

S

SZL HOLDINGS · SZL COMMAND

SUBSTACK LAUNCH PLAN · Q1 · COPY-PASTE READY

The **Substack** launch kit for SZL Command.

Positioning, 90-day calendar, 24 fully drafted essays, welcome flow, growth plan, monetization, KPIs, and setup checklist — everything needed to publish from day one.

Substack Launch Plan

Everything needed to launch SZL Command on Substack, in order. Every section is copy-paste ready. Pair this document with the Newsletter Screenshots zip for mockups and reference captures.

01	Positioning, Audience & Voice	03
02	Publication Identity & Bio	05
03	About Page (copy-paste)	07
04	Visual Identity Direction	09
05	Editorial Pillars & Share Mix	11
06	90-Day Editorial Calendar (24 essays)	12
07	Welcome Email / First Issue	14
08	Full Essay Library — all 24 posts, fully drafted	15
09	Growth Plan & Cross-posting Strategy	—
10	LinkedIn & X Amplification Templates	—
11	Monetization Plan & Founding-Member Offer	—
12	KPIs & Tracking Sheet Specification	—
13	Substack Setup Checklist (step-by-step)	—

Positioning, audience & voice.

Inside the build of SZL Holdings — a portfolio of AI-native command platforms for defense, maritime, real estate, legal, and advisory.

Target audiences

Investors

Vertical-AI and industrial-stack investors looking for a front-row seat on how a multi-product command group is built, funded, and operated.

What we publish for them: Investor updates in public · Portfolio deep-dives · Term-sheet and fundraising essays

Enterprise buyers

Defense program offices, shipowners / charterers / brokers, real-estate GPs and asset managers, heads of legal ops — the operators our products are built for.

What we publish for them: Vertical deep-dives (Aegis, Vessels, Terra, Prism Counsel) · Market structure essays · Compliance and governance essays

AI builders

Engineers and product leaders building production AI systems in regulated industries. Our engineering diary is for them.

What we publish for them: AI platform engineering essays · Eval / guardrail / agent-design posts · Incident post-mortems

Press & analysts

Journalists and analysts covering AI, defense-tech, maritime-tech, and industrial software. The newsletter is designed to be linkable and quotable.

What we publish for them: All posts — sources linked · Quarterly state-of-market pieces

Voice

- **Direct and specific.** Named products, named markets, named mistakes. No hedging for optics.
- **Operator-first.** Every essay is written for someone who could deploy the insight on Monday.

- **Numeric when possible, explicit when not.** Real metrics labeled as real; aspirational numbers labeled as such.
- **Calm.** No exclamation points, no hype, no "game-changer" vocabulary.
- **Seventh-grade readable, graduate-level substance.** Shortest sentence that carries the load.

Publication identity.

Publication name

SZL Command

szlcommand.substack.com · @szlcommand (X) · SZL Command (LinkedIn page)

Tagline

Command platforms, written in public.

Short bio (for profile fields, Twitter, LinkedIn)

Building SZL Holdings — a portfolio of AI-native command platforms across defense, maritime, real estate, legal, and advisory. Writing two long-form essays a week on the build, the market, and the lessons.

Long bio (for About page opener)

I build command platforms.

SZL Holdings is a portfolio of AI-native command platforms: Aegis (defense & intelligence), Vessels (maritime), Terra (real-estate intelligence), Carlota Jo (advisory), Command Portal (unified operator cockpit), CORTEX (mobile), IMPERIUM (governance), Forge (client delivery), Autopilot (agentic workflows), and Prism Counsel (legal & compliance).

Each product is a vertical operator surface with AI as an ingredient rather than the product. The spine underneath — auth, evals, guardrails, audit, governance — is shared. The bet is that the next decade of enterprise software will be won by small groups of vertical command platforms with a shared AI backbone, sold to specific operator roles, and compounding trust faster than horizontal AI vendors can flatten it.

This newsletter is where I write about building all of it — in public, in order, with the receipts. Two essays a week, six pillars, no fluff.

About page — paste verbatim.

About SZL Command

SZL Command is the writing home of the SZL Holdings build.

What you'll get

- Two long-form essays every week (Tuesday and Friday).
- Portfolio deep-dives, market structure analysis, engineering essays, and a monthly investor-style update published in public.
- Quarterly state-of-market pieces covering defense, maritime, and real estate from an operator-tech perspective.
- Occasional interviews with operators, investors, and builders in adjacent spaces.

The six pillars

1. **Defense & Intelligence** — Aegis, command-surface design, AI in high-assurance environments.
2. **Maritime** — Vessels, port-call intelligence, freight and bunker markets, compliance.
3. **Real Estate Intelligence** — Terra, underwriting, market structure, migration and climate.
4. **AI Platform Engineering** — evals, guardrails, agent design, infra decisions, incident learning.
5. **Founder Journey** — hiring, fundraising, governance, and running six products with a small team.
6. **Portfolio Deep-Dives** — one product per month, top to bottom.

Who reads this

Investors (vertical AI, industrial stack), enterprise buyers in defense / maritime / real-estate / legal, AI builders in regulated industries, and press covering the space. If you're one of those, you're the audience.

What you won't get

- Recycled news commentary.
- "Thought leadership" that's really a vendor pitch in a blazer.
- Fabricated numbers. When a metric isn't real yet, it's marked as such.

Paid tier

A founding-member tier is available. It includes:

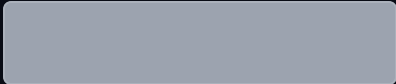
- The quarterly investor-style deep-dive memo (paid-only).
- A monthly small-group office-hours call.
- A private forum and directory.
- Every essay stays free — paid gets you the *extras*, not the essentials.

Contact

Reply to any email, or reach out via the SZL Holdings site.

Visual direction.

Palette: SZL Holdings — Platinum/Obsidian. Dark-first, gold as signal.

 SZL Base #060912 Primary background	 SZL Secondary #070a10 Panels, cards	 SZL Accent (Gold) #D4A054 Brand mark, headlines, CTAs
 SZL Accent Light #E3BB80 Hover, highlights	 Text Primary #F2EFE8 Body copy	 Text Secondary #9CA3AF Secondary copy
 Lyte Cyan #0FB3D4 Product signal (Lyte)	 Alloy Indigo #3A63E0 Product signal (Alloy)	 Aegis Blue #3B82F6 Defense/security mentions
 Vessels Sky #1D90D1 Maritime mentions	 Terra Green #40856A Real-estate mentions	 Carlota Amber #C8913A Advisory mentions

Typography

Display: Space Grotesk (weights 500/600/700) — used for headlines, section headers, pull quotes.

Body: Inter (400/500/600) — body copy, nav, lists.

Mono: JetBrains Mono (400/500) — metrics, code, eyebrow labels.

Exception: Cormorant Garamond reserved for Carlota Jo advisory posts only. Do not use elsewhere.

Cover image direction

Every cover image is one of three archetypes:

- **Topographic / Cartographic** — layered contour lines, maritime charts, parcel overlays, command-grid textures. Monochrome, gold accent line.
- **Data-surface** — abstract dashboard treatments with minimal data glyphs, glassmorphic depth, gold highlight on the KPI that matters.
- **Portrait / Object** — a single physical object or detail (a bridge, a port, a contract, a screen) shot or rendered against the obsidian palette with one gold highlight.

Never use stock AI-generated people or generic "technology" imagery. Never use more than one gold highlight per cover.

UI principles

- Dark-first, not dark-mode. The obsidian palette is the home, light is the exception.
- Gold is a signal, not a decoration. One per view.
- Typographic hierarchy does the heavy lifting — size, weight, tracking. Color comes last.
- All numbers are tabular mono. Operator trust requires numeric alignment.
- Whitespace is a budget line. Spend it generously on launch posts, tighter on updates.

Editorial pillars.

Each of the 24 posts below is fully drafted in the companion posts file (copy-paste ready). The cadence is two posts per week for 12 weeks = 24 essays = one full quarter. Week 13 carries the 90-day review as the final post of the arc. Every post includes Substack tags, Medium tags (five), and the LinkedIn and X amplification templates below.

#	PILLAR	SHARE	SIGNAL COLOR
1	DEFENSE & INTELLIGENCE	17%	Aegis Blue
2	MARITIME	17%	Vessels Sky
3	REAL ESTATE INTELLIGENCE	17%	Terra Green
4	AI PLATFORM ENGINEERING	21%	Lyte Cyan
5	FOUNDER JOURNEY	17%	SZL Accent (Gold)
6	PORTFOLIO DEEP-DIVES	11%	Alloy Indigo

90-day calendar.

Two essays per week · 12 weeks · 24 full drafts included (Section 08).

WEEK	PILLAR	TITLE	POST
W01	FOUNDER	Why I'm Building SZL Holdings in Public	#01
W01	AI	The Case for Vertical Command Platforms	#02
W02	DEFENSE	Inside Aegis: Building a Command Surface for Modern Defense	#03
W02	AI	Evals Are the Product	#04
W03	MARITIME	The Shipping Industry Is a Software Desert	#05
W03	FOUNDER	What I Got Wrong About B2B Sales in Year One	#06
W04	REAL	Inside Terra: Real Estate Intelligence for the Next Decade	#07
W04	PORTFOLIO	Command Portal: The Operator's Cockpit	#08
W05	FOUNDER	The First Five Hires in a Multi-Product Group	#09
W05	AI	What Actually Breaks in Production AI	#10
W06	PORTFOLIO	Inside Prism Counsel: The Quiet Backbone	#11
W06	FOUNDER	What I Look for in a Term Sheet	#12
W07	MARITIME	How We Rebuilt Port-Call Intelligence From the PDF Up	#13
W07	FOUNDER	My Weekly Operating Rhythm, In Detail	#14
W08	AI	Defensibility in Vertical AI	#15
W08	FOUNDER	The Monetization Ladder	#16
W09	FOUNDER	Cross-Posting Strategy: Substack, Medium, LinkedIn, X	#17
W09	PORTFOLIO	Forge: Client Delivery as a Product	#18
W10	FOUNDER	Q1 Investor Update, In Public	#19

WEEK	PILLAR	TITLE	POST
W10	AI	How I Decide What Becomes an Agent	#20
W11	MARITIME	State of the Commercial Maritime Market, Q1	#21
W11	FOUNDER	What I Learned From the One Pilot We Lost	#22
W12	PORTFOLIO	CORTEX: Why Command Platforms Need a Mobile Surface	#23
W13	FOUNDER	90 Days of Building in Public	#24

Welcome email / first issue.

Subject

Welcome to SZL Command — what's here, and what's next.

Preview text

Two essays a week, six pillars, written from inside the build.

Body (copy-paste)

Hi — thanks for subscribing to **SZL Command**.

You just bought a front-row seat on the build of SZL Holdings — a portfolio of AI-native command platforms for defense, maritime, real estate, legal, and advisory. Here's what you can expect.

The rhythm

- **Tuesday** — a long-form essay on one of the six pillars (Defense & Intelligence, Maritime, Real Estate, AI Platform Engineering, Founder Journey, Portfolio Deep-Dives).
- **Friday** — the second essay of the week. Usually product-or-market, sometimes the founder journey.
- **Quarterly** — an investor-style update, published in public, with the same numbers I send the cap table.

Three places to start

1. **Why I'm Building SZL Holdings in Public** — the founding post. The *what* and the *why*.
2. **The Case for Vertical Command Platforms** — the thesis behind every product in the portfolio.
3. **Inside Aegis: Building a Command Surface for Modern Defense** — the first product deep-dive.

One ask

Reply to this email and tell me one thing: which of the six pillars do you want me to prioritize in the next 30 days? I read every reply and the answers shape the calendar.

A note on the paid tier

The paid (founding-member) tier is open. It includes the quarterly investor-style deep-dive memo, a monthly small-group office-hours call, and a private forum and directory. Every essay on the list stays free — paid is for the *extras*, not the essentials.

If you're an investor, buyer, builder, or operator in one of the domains I cover, you are exactly the person this newsletter is written for. Welcome in.

— S

P.S. — If someone you respect forwarded you this, you can subscribe directly at szlcommand.substack.com.

24 fully drafted essays.

Every essay below is copy-paste ready. Order matches the 90-day calendar (two posts per week for twelve weeks).

SUBSTACK USAGE NOTES

Each essay carries two tag rows at the end: **Substack tags** (shorter, lowercase, 3–5) and **Medium tags** (pick any 5). Publish on Substack first; cross-post to Medium 24 hours later with canonical pointing back to the Substack URL. The Medium version of this kit handles the inverse.

CONTINUE TO ESSAY 01 →

Why I'm Building SZL Holdings in Public

A command-platform group for defense, maritime, real estate, and the AI layer underneath all of it.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

I've spent the last few years building something that doesn't fit cleanly into a single category. It isn't a SaaS product. It isn't a consultancy. It isn't a fund. It's closer to what used to be called an *operating group* — a small portfolio of AI-native command platforms that share a spine.

The holding company is called **SZL Holdings**. The platforms sit underneath it: **Aegis** for defense and intelligence, **Vessels** for maritime, **Terra** for real estate, **Carlota Jo** for executive advisory, **Command Portal** as the unified operator cockpit, **CORTEX** as the mobile surface, **IMPERIUM** as the governance layer, **Forge** for client delivery, **Autopilot** for agentic workflows, and **Prism Counsel** for legal and compliance tooling. Each one exists because an operator somewhere is running a critical function on spreadsheets, PDFs, and hope.

This newsletter is where I'll write about building all of it — in public, in order, with the receipts.

Why a newsletter, and why now

There are three reasons.

First, the work is more legible when it's written down. I've built enough of these systems now that patterns are emerging — how command surfaces should behave, where AI belongs and where it doesn't, how to make an enterprise-grade tool feel like a consumer one. If I don't write those patterns down, I keep re-deriving them.

Second, the people I want to meet read. Investors who understand vertical AI, defense primes with modernization budgets, maritime operators tired of port-call PDFs, real estate GPs who want real intelligence instead of CoStar screenshots — these people don't scroll TikTok looking for B2B software. They read long-form. They subscribe. They forward.

Third, founders who build in public compound faster. The feedback loop is shorter. The mistakes are cheaper because they happen on a draft page instead of inside a live deal. And the relationships formed through the writing — with operators, writers, funders — tend to outlast any single product.

What this newsletter is, concretely

Two posts a week. Tuesday and Friday. No fluff, no "thought leadership" that's really just a vendor pitch in a blazer.

The pillars:

1. **Defense & Intelligence** — what I'm learning from building Aegis, where AI belongs in the kill chain (and where it doesn't), how to ship software that survives an accreditation review.

2. **Maritime** — why the shipping industry is a software desert, how Vessels is closing the gap, market structure pieces on freight, ports, and class societies.
3. **Real Estate Intelligence** — Terra's bet: that the next decade of real estate returns is won by operators with better data, not better instincts. Zoning, migration, rent pressure, climate.
4. **AI Platform Engineering** — the unsexy infrastructure decisions: evals, guardrails, the difference between a demo and a deployable system, how I ship agentic features without blowing up trust.
5. **Founder Journey** — hiring, fundraising, governance, what actually works when you're running six products at once with a small team.
6. **Portfolio Deep-Dives** — one product per month, top-to-bottom: problem, architecture, unit economics, roadmap, what I got wrong.

Who this is for

If you're an **investor** looking at vertical AI, this is a front-row seat to what it looks like to build across domains, where the defensibility actually sits, and what the real rather than the pitched numbers look like.

If you're an **operator** in defense, maritime, real estate, or advisory, this is the place to see what the command surfaces your competitors will have in 24 months look like today — and to send me the workflows you want next.

If you're a **builder**, this is the engineering diary I wish someone had kept when I was learning how to ship AI-native systems that grownups would actually pay for.

If you're a **writer or journalist** covering AI, defense tech, or the new industrial stack, this is the source document. I'll link everything, show the work, and make it easy to quote.

What I'll promise

- I won't publish anything I wouldn't put in a board deck.
- I won't fake numbers. If a metric isn't real yet, I'll say so.
- I won't pretend to have solved problems I'm still actively bleeding on.
- I'll link to GitHub and live demos wherever I can.
- I'll keep the free tier generous. A paid tier is coming — it'll include deeper portfolio deep-dives, the quarterly investor memo, and a monthly office-hours call — but the essays stay free.

What you can do right now

Three things:

1. **Subscribe.** If you've read this far, you're the audience. The button is at the top and the bottom.
2. **Forward this** to one operator, one investor, or one builder who should be reading along.
3. **Reply.** Tell me which of the six pillars you want more of, and what you'd like me to write about first inside it. I read everything.

Next post drops Friday: *The Case for Vertical Command Platforms (and Why Horizontal AI Is Eating Itself)*.

Welcome aboard.

SUBSTACK TAGS:

FOUNDER

AI

STARTUPS

BUILDING-IN-PUBLIC

MEDIUM TAGS (PICK 5):

STARTUP

ARTIFICIAL INTELLIGENCE

ENTREPRENEURSHIP

BUILDING IN PUBLIC

TECHNOLOGY

The Case for Vertical Command Platforms

Horizontal AI is eating itself. The next decade of software belongs to operator-grade, domain-specific command surfaces.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

There's a fight happening right now that most people are watching from the wrong angle.

On one side: the frontier labs and horizontal AI platforms — OpenAI, Anthropic, Google, and every wrapper built on top of them. Generic chat, generic agents, generic "copilot for your work." Massive distribution, enormous capital, and a fundamental problem they don't talk about on earnings calls: **margin compression is coming for all of them.**

On the other side: a quieter movement of *vertical command platforms* — software built for one specific operator role, in one specific domain, with AI as an ingredient rather than the product. These are the boring, lucrative, defensible companies that most AI-bull narratives skip right over.

I'm building on the vertical side, on purpose. Here's why.

Horizontal AI is a commodity in slow motion

The frontier models are converging. GPT-class, Claude-class, Gemini-class, and the open-weights tier are all within a couple of percentage points on most evals that matter for production work. Cost per token drops 60–90% per year. The "moat" that horizontal vendors sell — raw capability — erodes every time a competitor ships a better base model or a better fine-tune.

That's not a dig. It's just the history of infrastructure. Databases, cloud compute, CDN, even search — the base layer commoditizes. The money moves up the stack.

The trouble for horizontal AI is that the layer immediately above them — general-purpose assistants, "copilot for everything" — is the *worst* place to build a durable business. The user has to explain their job every single time. The system has no memory of the workflow. Integrations are shallow. Security teams hate it. And the switching cost is roughly one browser tab.

What a vertical command platform actually is

A command platform is software that takes a specific operator — a fleet manager, a defense analyst, an asset manager, a head of client services — and gives them **a single surface that does their whole job.**

It has four properties generic AI doesn't:

1. **A domain data model.** Not a generic document store — an actual schema for vessels, berths, bunkers, and port calls. Or for parcels, zoning overlays, rent rolls, and comps. Or for targets, indicators, and collection plans. The vocabulary is the product.
2. **Workflow-shaped UI.** The operator doesn't ask the system what to do. The system already knows the top ten things that matter today, in priority order, and lets the operator act on them in a few clicks.

3. **AI as an ingredient, not the experience.** Generation, extraction, classification, summarization, and agentic execution happen *inside* workflows — not as a separate chat window the user has to remember to use.
4. **Compliance and trust built in from day one.** Role-based access, audit logs, redaction, tenant isolation, evals, reviewer queues. Not because it's fun, but because no serious buyer will write a check without them.

Aegis, Vessels, Terra, and Carlota Jo are each a command platform for one of these operator roles. Command Portal is the *unified* command surface for someone running multiple of them — an operator of operators.

Why this shape wins

Three reasons.

1. The margins are real

Horizontal AI vendors quietly run sub-50% gross margins when you include inference costs at scale. Vertical command platforms run 75–85% gross margins at maturity, for the same reason Bloomberg does: the software is the small part of the cost, the data model and workflow are the value, and the customer will cheerfully pay five figures a seat because it replaces three people and a consulting retainer.

2. The moat compounds

Every workflow you absorb into the platform — every port-call ritual, every intel packet, every asset-management decision — becomes a retention hook. Users who run their job inside your surface don't switch. They extend.

This is the Bloomberg lesson, the Salesforce lesson, the Procore lesson, the Veeva lesson. It's also the Palantir lesson, which is the one most AI bulls miss: Palantir isn't "AI at work," it's an operating system for a specific operator that happens to use AI inside it.

3. The distribution is cheaper

Horizontal AI products fight for attention in the world's most crowded feed. Vertical platforms sell to identifiable operators — there are maybe 2,000 global maritime fleet managers who matter, maybe 5,000 defense modernization program offices, maybe 10,000 real estate GPs worth selling to. You can build a list. You can show up. You can write a newsletter they actually read.

(Hi.)

The SZL bet, in one paragraph

The next decade of enterprise software will be won by small groups of vertical command platforms that share an AI platform spine, sell to specific operator roles, and compound trust faster than the horizontal vendors can flatten it. SZL Holdings is one of those groups. Aegis, Vessels, Terra, and Carlota Jo are the commands. Command Portal, CORTEX, IMPERIUM, Forge, Autopilot, and Prism Counsel are the spine. The newsletter is the scoreboard.

What to watch for

If the thesis is right, you'll see three things over the next 24 months:

- **Horizontal AI tools will start vertical-izing**, badly. Expect "Copilot for Maritime" style launches that ship a generic chat wrapper over a weak data model. They will not stick.
- **Vertical command platforms will start consolidating**, because the AI spine underneath is expensive to build well and the gross-margin profile rewards groups over point products.
- **Buyers will get better at telling the difference**. RFPs will start asking about evals, audit logs, redaction, and domain data models by name, not just features.

I'll write about each of those as they happen, with live examples from inside the portfolio.

If this resonates, forward it to the operator or investor in your life who's still on the "AI will do it all" side of the aisle. And subscribe — next week I'm going inside Aegis.

— S

SUBSTACK TAGS:

AI

VERTICAL-SAAS

PLATFORMS

MEDIUM TAGS (PICK 5):

ARTIFICIAL INTELLIGENCE

SAAS

ENTERPRISE SOFTWARE

PRODUCT DESIGN

TECHNOLOGY

Inside Aegis: Building a Command Surface for Modern Defense

What it takes to ship software that survives accreditation, briefings, and 3am alerts.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

Aegis is the SZL platform for unified defense and intelligence command. If you want the short version: it's what a modern J2/J3 shop should have been using for the last ten years — fused signals, structured analyst workflows, AI-assisted tradecraft, and a command surface that doesn't look like it was designed in 2004.

This post is the long version.

The problem Aegis solves

Modern defense and intelligence work has a structural mismatch. The threat environment is real-time, multi-domain, and increasingly autonomous. The tooling most analysts and operators actually use is not.

Three patterns show up everywhere I look:

1. **Signal fragmentation.** ISR feeds, HUMINT reporting, OSINT, partner sharing, commercial imagery, and cyber telemetry live in different systems with different access controls. Fusion happens in a PowerPoint.
2. **Tradecraft drift.** Good analysts have sharp internal processes — collection planning, hypothesis testing, confidence scoring, RFI management. Most tools treat those as afterthoughts. The process lives in the analyst's head or in a Word template.
3. **Command latency.** By the time a fused picture reaches a decision-maker, the window has often closed. Not because the analysts are slow — because the *surface* between analyst and commander is a series of slide decks.

Aegis attacks all three.

The shape of the product

Aegis is organized around four surfaces:

1. **The Common Operating Picture.** A live map-plus-timeline view fused from every connected signal. Not a dashboard — a workspace. Operators can pin, annotate, and build a case directly on the picture. AI assists on entity resolution, pattern-of-life detection, and anomaly flagging, with every inference tagged to the underlying evidence.
2. **The Analyst Workbench.** Structured collection plans, RFI queues, hypothesis trees, and a confidence-scored report builder. Writes its own draft products and links every claim to source evidence. Reviewers see a diff. Nothing ships without a human in the loop.

3. The Command Brief. A commander-facing surface that auto-generates the current picture, the top decisions needing input, and the recommended courses of action with confidence and dissent captured. This is the layer most tools skip — and it's the one that actually gets used at 0600.

4. The Governance Spine. Role-based access, compartment isolation, full audit log, redaction on export, prompt and output logging for every AI call, and an evals harness that runs on every deploy. Boring. Non-negotiable.

Where AI belongs — and where it doesn't

This is the part I get asked about most.

AI belongs in entity extraction from unstructured reporting, cross-source linking, translation, transcription, anomaly detection in telemetry, first-draft report generation, and collection-plan suggestion. These are tasks where the cost of a wrong answer is bounded, the human reviewer is cheap and fast, and the productivity gain is enormous.

AI does not belong in targeting recommendations without human-in-the-loop, in confidence scoring of its own outputs (a model assessing its own reliability is a closed loop), in any workflow that touches lethal authority without an explicit operator decision captured and logged, or in anything where the training data provenance isn't auditable.

Every Aegis AI feature ships with three things: the prompt, the evals, and the dissent path. If an analyst disagrees with an AI output, that dissent is a first-class object — not a comment buried on a page.

The hard parts nobody puts in the demo

Accreditation. Shipping into a classified environment is not a faster version of shipping into a commercial one. Every dependency, every model, every log path is a line item someone has to sign. We designed Aegis around a cleanly separable core that can run in a disconnected enclave with local inference, because the cloud-only version of this product is unsellable to the customers who need it most.

Eval infrastructure. The thing that keeps me up at night isn't model capability — it's model drift. A 2% regression in extraction accuracy across a quarterly model update is invisible in casual use and catastrophic in an intel product. Aegis has a red-team eval suite that runs every night on gold-labeled historical cases, and any degradation pages me.

Tradecraft capture. Good analysts do not want their craft "automated." They want it amplified. The product discipline is to observe how a senior analyst actually works, encode the ritual as a first-class object (the collection plan, the hypothesis tree, the RFI thread), and then let the AI sit *inside* that ritual instead of replacing it. Every feature we shipped that tried to skip the ritual got rejected by the users who mattered.

What's shipped, what's coming

Today Aegis has the common operating picture, the analyst workbench, the command brief, and the governance spine running against simulated and commercial data. We're in active conversations with two program offices and one allied partner about pilot deployments into real environments.

Next six months: disconnected enclave mode, deeper OT/ICS ingest (more on that in a future post), and a dedicated CISO executive view that aggregates the security modules into a single cross-domain KPI dashboard.

Why I'm writing about it

Two reasons. One, because defense software that's built in public — with the hard parts named — is still rare, and I want more of it to exist. Two, because the people who should be building and buying this next generation of platforms are mostly reachable through writing, not through trade-show booths.

If you're in a program office, a prime, an allied service, or an investor focused on the defense stack, reply to this email. I'll share the live walkthrough and the brief on the accreditation path.

Next post: how Vessels is doing the same thing for the global shipping industry — a market that's two decades behind defense on tooling and waking up fast.

— S

SUBSTACK TAGS:

DEFENSE

AEGIS

COMMAND-PLATFORMS

MEDIUM TAGS (PICK 5):

DEFENSE TECHNOLOGY

ARTIFICIAL INTELLIGENCE

NATIONAL SECURITY

PRODUCT DESIGN

GOVERNMENT

Evals Are the Product

Why the eval harness is the most underrated piece of any production AI system — and what mine looks like.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

Here is the single highest-leverage piece of advice I can give anyone shipping AI features into production: **the eval harness is the product.**

Not the model. Not the prompt. Not the RAG pipeline, the agent loop, the fine-tune, or the fancy function-calling schema. The eval harness — the thing that tells you, repeatably and cheaply, whether your system got better or worse today — is the asset that compounds.

Most teams treat it as an afterthought. That's why most teams are one model update away from a silent regression that eats their trust.

Here's how I think about it, and what the harness looks like inside the SZL stack.

Three uses of evals

Teams conflate three very different things and call them all "evals." They aren't the same.

1. Capability evals. Does the model, in isolation, have the ability to do the task? This is what MMLU, HumanEval, and GSM8K measure. Useful for model selection. Almost useless for production confidence.

2. System evals. Does my *system* — model + prompt + retrieval + tool use + guardrails — produce the right answer for my domain? This is what actually matters. Nobody else can build this for you.

3. Regression evals. When I change the model version, the prompt, the retrieval strategy, or the system prompt, did anything get worse? This is the one that runs in CI and pages you at 2am.

Most teams build capability evals (easy, visible), skip system evals (hard, invisible), and only find out they need regression evals after a production incident.

What a good eval set looks like

Four properties:

- **Gold-labeled by domain experts**, not by the same model you're testing. If an analyst labels an extraction task, the analyst is ground truth. LLM-as-judge is fine for directional signal, but it must never be the only signal on anything consequential.
- **Representative of real traffic**, including the long tail. Sample from production. Anonymize. Keep the hard ones.
- **Stable over time.** The set should not drift. You want to compare July to October on the same questions, not a constantly-moving target.

- **Small enough to run every commit.** If it takes four hours to run, nobody runs it. Mine are tiered: a 50-case smoke suite on every commit, a 500-case full suite nightly, a 5,000-case quarterly audit.

What we measure

For each product in the portfolio, the eval axes are different, but the pattern is the same:

- **Accuracy** on the core task (extraction, classification, generation, retrieval).
- **Faithfulness** — does the output cite only what's in the source?
- **Format adherence** — does the structured output match the schema, every time?
- **Refusal correctness** — does the system refuse when it should, and not when it shouldn't?
- **Latency and cost** at p50 and p95.
- **Safety** on red-team prompts specific to the domain. For Aegis that's different from Terra, which is different from Prism Counsel.

Every axis has a green/yellow/red threshold. Any red on any axis blocks the deploy.

How I run them

A small homegrown harness. Nothing exotic. The important properties:

1. Every eval case has an ID, a version, an expected output, and a set of acceptable-variant outputs. No "vibes-based" pass/fail.
2. Results are stored per git SHA, not per "day." I can diff eval results between any two commits.
3. Regressions trigger a structured report, not a Slack bell. The report names the cases that moved, shows the old and new outputs, and tags the likely cause.
4. Human review queues exist for anything the harness flags as ambiguous. Humans label, the labels feed back into the set.

I have been through four eval tools and rolled our own three times. The harness is always the cheapest part of the system to build and the most expensive part to ignore.

The meta-point

Building a production AI system without an eval harness is like shipping code without tests — except worse, because the model changes underneath you. A team with mediocre prompts and great evals beats a team with great prompts and no evals every single quarter.

The easy heuristic: **if you can't tell me whether your system is better this week than last week, you don't have a product. You have a demo.**

Two action items if you're building:

- Write down ten real examples of what your system should do, and ten of what it should refuse. That's version one of your eval set. Today.
- Put those in CI. Fail the build if accuracy on the ten drops. That's version one of your regression gate. This week.

You will thank yourself in six months. And your buyers — whoever they are — will pay more.

Next post: why the shipping industry is a software desert, and how Vessels is closing the gap.

— S

SUBSTACK TAGS: AI EVALS ENGINEERING

MEDIUM TAGS (PICK 5): ARTIFICIAL INTELLIGENCE SOFTWARE ENGINEERING MACHINE LEARNING PRODUCT DEVELOPMENT TESTING

The Shipping Industry Is a Software Desert

90% of global trade runs on software built before the iPhone. Vessels is the correction.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

If you want to see what the enterprise software market looked like in 2005, go visit a shipping company.

I don't mean this as a dunk. Maritime is one of the most operationally sophisticated industries on earth — it moves 90% of global trade, runs on margins you'd consider a rounding error in tech, and has a safety record that's the envy of every other heavy industry. The people are excellent. The tooling is not.

This is the market Vessels is built for. This post is why.

The state of the art, honestly

Walk into a typical fleet operations center today and you will see:

- **Three to seven vendor systems** held together by Outlook. Voyage management in one tool, bunkering in another, crewing in a third, port-call management in Excel, commercial chartering over WhatsApp.
- **Port calls managed over email and PDF.** The Statement of Facts — the document of record for what actually happened during a call — is often retyped from a PDF into a spreadsheet into a database. Three times, per call.
- **Compliance by screenshot.** Sanctions screening, emissions reporting (CII, EU ETS, FuelEU Maritime), and cargo documentation are reconciled manually against multiple public and private data sources.
- **Commercial decisions by instinct.** A \$30M vessel is often chartered on the basis of a 20-minute phone call and a rate gut-check, because the benchmarking data exists in Baltic Exchange PDFs and brokers' heads, not in a decision surface.

None of this is anyone's fault. The incumbents were built in the era before broadband, and the switching cost of ripping them out is genuinely high. But the structural opportunity is enormous, and it's wide open.

What changed

Three things in the last thirty-six months.

1. AIS is suddenly good enough. Terrestrial and satellite AIS coverage has converged to the point where you can have a global real-time picture of every vessel over 300 gross tons, with historical replay, for a price a mid-sized operator can afford. That was not true five years ago.

2. Language models made unstructured data tractable. Every port call produces a pile of PDFs, PDAs, Notices of Readiness, and email threads in forty different templates from forty different agents. LLMs

make extracting a structured port-call record from that mess a solved problem. The incumbents haven't shipped it yet.

3. Compliance got serious. EU ETS, FuelEU, CII, and the next round of sanctions regimes made "we'll do it in Excel" a board-level risk, not an ops-team inconvenience. The software-buying authority has moved up the building.

What Vessels is

Vessels is a unified commercial and operational command surface for fleet operators, charterers, and shipowners. Seven integrated modules:

1. **Live fleet picture.** Map, timeline, eventing, fused AIS and internal systems.
2. **Port-call intelligence.** Auto-extracted Statements of Facts, demurrage/despatch calculations, agent performance benchmarking.
3. **Voyage P&L.** Real-time profit-and-loss per voyage, with freight-rate benchmarking against live market indices.
4. **Bunker optimization.** Price, quality, and location optimization with emissions impact.
5. **Compliance command.** CII, EU ETS, FuelEU Maritime, sanctions screening, cargo documentation — one surface, full audit trail.
6. **Commercial intelligence.** Rate benchmarking, counterparty risk, tonnage positioning, opportunity detection.
7. **Safety and incident management.** Near-miss capture, incident workflows, class and flag reporting.

AI sits inside each module — extraction on port calls, anomaly detection on AIS, forecasting on rates, summarization on compliance packs — but the UI is operator-first, not chat-first.

The bet

The bet is not that Vessels will displace IMOS or Veson overnight. Those are good products with deep roots. The bet is that **the next generation of operators** — the ones taking over fleets from their parents, the ones running newly constituted pools, the ones rolling up mid-sized operators into larger ones — will buy a unified, AI-native command surface because the incumbents cannot ship one at their pace.

I've sat with enough of them now to be confident about this. The question isn't *whether* — it's *who*.

Why write this out

Two reasons. One, because the maritime trade press is full of press releases but short on product thinking, and I want to contribute to the signal. Two, because the buyers, partners, and builders in this space read long-form far more than any other B2B audience I've encountered. Port agents, chartering desks, and shipowners read. They read on ships, they read in hotels, they read on flights. This is where they are.

If you're in maritime — operator, broker, agent, class, flag, charterer, shipowner — reply. I'll send the walkthrough. If you're a writer or journalist covering the space, same offer with better visuals.

Next post: what I got wrong about B2B sales in my first year running a portfolio.

What good looks like

A modern operator-grade maritime stack should give a charterer or shipowner four things on one screen, in real time, with audit trails attached.

Position truth. Not the AIS feed alone — the AIS feed reconciled against satellite-confirmed positions, port-call records, and the vessel's last reported heading and speed. When an asset's transponder goes dark in the South China Sea or the Gulf of Aden, the operator should know within minutes, with a confidence-scored last-known-position estimate and a flagged anomaly alert.

Voyage economics, live. Bunker price by port, ETA-adjusted laytime exposure, demurrage clock, port dues, agent fees, and the weather-routed time-charter equivalent — all updating against the actual voyage in progress, not against the assumptions in the original fixture note.

Counterparty signal. When a charterer fixes a vessel, the operator should see the charterer's payment history, current open positions, sanctions-screening status, and the historical performance of the brokers in the chain. Today this is tribal knowledge held by three brokers in London, Singapore, and Athens.

Compliance state. EU-ETS exposure, CII rating projection, MARPOL Annex VI compliance, sanctions screening on every counterparty in the voyage, and document chain-of-custody for the bills of lading. One panel, structured, exportable for audit.

That's the bar. Vessels is built to that bar.

Why incumbents won't catch up

Three reasons, in order of difficulty.

First, the data plumbing. Reconciling AIS, port-call records, and document chains across a hundred jurisdictions is a multi-year engineering project, and the incumbents are committed to the schemas they shipped in 2003. They can't rewrite without breaking customers.

Second, the operator surface. Maritime operators have learned to expect bad software, and the new entrant who shows up with a 2004 dashboard with three more buttons doesn't move the needle. The bar is iPhone-grade, and the people who can build that aren't the people the incumbents can hire on the salary bands they have.

Third, the trust layer. The buyers are conservative for excellent reasons — a wrong number on a charter party is a million-dollar mistake. Trust is earned by being correct in production for two years, not by a glossy demo. The incumbents have decades of "correct enough" on their side. The challenger has to be visibly more correct, with the audit trail to prove it.

The good news for builders: the customers know the incumbents are leaving money on the table. Every shipowner I've spoken to in the last twelve months has a story about a six-figure number that fell through a software crack. That's the wedge.

SUBSTACK TAGS:

MARITIME

SHIPPING

VESSELS

MEDIUM TAGS (PICK 5):

LOGISTICS

SUPPLY CHAIN

MARITIME

ENTERPRISE SOFTWARE

TECHNOLOGY

What I Got Wrong About B2B Sales in Year One

Five expensive lessons from pitching command platforms to buyers who don't buy the way you think they do.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

I walked into year one of SZL Holdings thinking I knew how B2B software got sold. I had read the books, worked with good salespeople, and shipped enough product to believe the product-led motion applied to almost everything.

I was wrong about a lot of it. Here are the five expensive lessons, in the order I learned them.

1. "Decision-maker" and "buyer" are different people.

The decision-maker is the person whose problem you solve. The buyer is the person with the budget. In mid-market they're often the same human. In the kinds of accounts SZL sells into — defense program offices, shipping groups, real estate GPs — they are almost never the same.

The consequence: if you run a great demo with the decision-maker and skip the buyer, you get "this is amazing, let me loop in procurement" and then silence for four months. If you run a great demo with the buyer and skip the decision-maker, you get an RFP written against someone else's product.

Fix: Map the account before the first call. Decision-maker, buyer, economic champion, technical reviewer, security reviewer, legal reviewer. Six names minimum. Anything missing is a risk.

2. Enterprise buyers do not buy features. They buy *risk reduction*.

Every founder wants to demo the slickest, most ambitious part of the product. Enterprise buyers want to know what happens when it breaks. Who they call. Whether their job is at risk. Whether their auditors sign off.

The slickest demo I ever gave in year one lost. The reason it lost, per the buyer: "we can't tell who's accountable when your agent does something wrong."

Fix: Lead with governance. Show the audit log, the role-based access, the eval harness, the human-in-the-loop review queue — *before* you show the headline feature. Boring wins in enterprise.

3. The pilot is the product.

First-year me ran pilots like proofs of concept — hand-holding, scoped down, "let's just show value." That produced pilots that were impressive and not renewable.

Pilots should be miniature versions of the real deal. Same data shape, same users, same governance, same integration surface. If the pilot works, the production deployment is a scaling exercise, not a replatform.

Fix: Define success metrics, deployment footprint, and exit criteria before signing. If you can't write the production rollout plan on day one, the pilot isn't ready.

4. Sales cycles are set by the buyer's calendar, not yours.

Government buys on fiscal year. Maritime buys around budget windows tied to vessel cycles. Real estate buys against fund-deployment timelines. Defense buys against program milestones.

Trying to close a deal "this quarter" when the buyer's next meaningful approval window is Q3 next year is not hustle. It's noise.

Fix: Ask, on the first real call, "when is your next funding / approval / procurement window for this category?" That is the deal timeline. Everything before it is relationship building.

5. Champions are not free. Protect them.

In every closed deal, there is a person inside the buyer's organization who fought for you. That person burned internal capital to do it. If you don't give them cover — a strong reference, a clean deployment, a sharp ROI memo they can forward — they never fight for you on the next deal.

First-year me treated champions as information sources. Second-year me treats them as the single most valuable relationship in the account.

Fix: Every champion gets a quarterly success package — an ROI memo written for their internal audience, not yours; a heads-up on upcoming features they can preview internally; an invitation to a small, curated customer council where their peers show up.

The meta-lesson

The five lessons rhyme: **enterprise buyers buy trust, not technology**. The product has to be excellent — the trust is the thing that converts excellence into revenue.

I'll write more sales-operations pieces as the year goes on, including a full post on how I've structured the discovery-to-pilot-to-production motion across three very different verticals (defense, maritime, real estate) with one small team.

If you're running a similar motion and want to compare notes, reply. These lessons compound faster when they're shared.

Next post: inside Terra — why I think the next decade of real estate returns is won by operators with better data.

— S

What I'd tell my year-one self

Three things.

One: write the contract before you write the deck. I lost two months on a deal because the buyer loved the demo and the deck and then discovered, in legal review, that our DPA didn't cover their data residency requirement. We could have known this in week one. Now every enterprise conversation

starts with a one-page term sheet — not to negotiate it, but to surface the dealbreakers before either side spends real time. The deals that survive the term sheet are the deals worth pursuing.

Two: hire the customer-success person before the third sale. Not after — before. The first two enterprise customers will tolerate scrappiness. The third one will not, because by then your reputation precedes you and they expect a real onboarding. The CS hire is also the person who tells you what's actually breaking in production, which is the most valuable signal you'll get all year.

Three: stop pricing on the calendar. I priced our first three pilots at "\$X for the next six months" because I wanted to lock in the relationship. What I locked in was a six-month delay on every pricing learning. Price per unit of value — per seat, per asset, per workflow — from the first paid pilot. If the customer doesn't want to commit to volume, that's a useful signal too.

The metric I now watch

One number, weekly: time from first qualified call to signed paid pilot. In year one we ran 90+ days. In year two we're targeting 30 days. The compression isn't from being pushier — it's from removing the dead time between meetings, pre-loading the legal artifacts, and being explicit that we don't do unpaid evaluations longer than two weeks. The customers who can't move at that pace are not customers we can serve well anyway.

The deals we lose on speed, we lose to nobody. They simply don't happen. That's a healthier outcome than a six-month courtship that ends in a "no."

SUBSTACK TAGS:

FOUNDER

SALES

B2B

MEDIUM TAGS (PICK 5):

SALES

ENTREPRENEURSHIP

STARTUP

BUSINESS STRATEGY

LEADERSHIP

Inside Terra: Real Estate Intelligence for the Next Decade

The operators who win the next cycle will have data surfaces the incumbents can't match.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

Real estate is a data industry that has been in denial about being a data industry for thirty years.

Ask any LP what drives fund returns and they'll quote you instincts, relationships, and deal flow. Ask any GP and they'll quote you the same, plus a grudging nod to market data. Then ask to see the dashboard the GP actually runs against — and you'll get a CoStar screenshot, a Costar export glued to an Excel model, and a PDF of last month's operator report.

There is a generation of operators coming up who are going to eat that lunch. Terra is the surface they'll use to do it.

The thesis in one paragraph

The last real-estate cycle was won by operators with cheap capital. The next cycle will be won by operators with **better data surfaces, better workflow tooling, and AI-assisted underwriting that compresses the time-to-no from days to minutes**. Capital cost isn't an edge anymore — everybody has access to the same rates. The edge is in how fast and how well you can evaluate a thousand parcels to pick the ten worth chasing.

What Terra does

Terra is a real estate intelligence command surface built for the GP, the asset manager, and the acquisitions team. Five surfaces:

- 1. Market intelligence.** Parcel-level overlays of zoning, entitlements, rent pressure, migration flows, climate risk, school and employment data, comps, and ownership history. Not a screenshot — a workspace where you can query, filter, and save pipelines.
- 2. Pipeline command.** Deal flow from source to kill-or-close, with AI-assisted screening against your fund's thesis. A sourcing partner sends you a CIM at 11pm; by 7am Terra has a one-pager with your thesis fit, the comp set, the risks flagged, and the three things to verify.
- 3. Underwriting workbench.** Structured underwriting with versioning, scenario analysis, and assumption tracking. Comparable to the Excel model in rigor, but with auditability that Excel will never have.
- 4. Asset management.** Real-time portfolio view, rent roll intelligence, operator benchmarking, capex tracking. Connects to property-management systems where they exist, extracts from PDFs where they don't.

5. LP and IC surfaces. Clean, on-brand reporting to LPs. Structured investment committee packets generated from the underwriting workbench, not hand-assembled at midnight before the meeting.

Where AI actually helps

The temptation in real estate AI is to sell a magic underwriting model. I am not going to do that, and neither is anyone else with a real product. The places AI helps in practice:

- **Document extraction.** CIMs, offering memos, rent rolls, operator reports, leases — turning PDFs into structured data at scale. Hours to seconds.
- **Thesis-fit screening.** Given a GP's written investment thesis, score inbound opportunities against it with an explainable breakdown.
- **Comp set construction.** Suggest comparable assets for a given property with reasoning. Human still picks the final set.
- **Risk flagging.** Climate, environmental, title, tenant concentration, lease roll — surface the flags the analyst would find on page 40 of the memo, on page 1.
- **Narrative generation.** First-draft IC memos, LP updates, and quarterly reports from the structured data. Humans edit.

What AI does not do in Terra: price the deal, replace the analyst's judgment, or sign anything.

The competitive map, honestly

The incumbents — CoStar, CompStak, RCA, Altus — are real companies with real data moats. Terra is not trying to be a data provider. Terra is a **surface** that integrates the buyer's internal data with licensed external data, wraps workflow and AI around it, and makes the GP's operation 3–5x more efficient per head.

That's a different product category. The closest analog is what Palantir did for defense — you don't beat the data provider, you sit on top of them and own the operator.

Who this is for

Three buyer profiles:

1. **Mid-market GPs** (\$200M–\$5B AUM) who are outgrowing spreadsheets but too small for bespoke enterprise tooling.
2. **Family offices** running direct real-estate programs who want institutional-grade process without institutional-grade headcount.
3. **Asset managers at large platforms** running a specific strategy (multifamily, industrial, net lease) who want a dedicated command surface for their book.

I am not selling to the three biggest institutional platforms. They will build their own, eventually, badly, and switch to Terra in year three.

What's next

The near-term Terra roadmap: deeper integration with the five major property-management systems, expanded climate-risk modeling (physical and transition), and a dedicated IC-packet workflow that

compresses the week-before-IC ritual from forty hours to four.

If you're a GP, an asset manager, or an LP curious about what the next generation of operator tooling looks like, reply. I'll send the walkthrough and the one-pager on the underwriting workbench.

Next post: the Command Portal — what it looks like when you unify six command platforms into one operator cockpit.

— S

SUBSTACK TAGS:

REAL-ESTATE

TERRA

INTELLIGENCE

MEDIUM TAGS (PICK 5):

REAL ESTATE

ARTIFICIAL INTELLIGENCE

INVESTING

DATA SCIENCE

TECHNOLOGY

Command Portal: The Operator's Cockpit

What happens when six vertical command platforms share one unified surface.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

The Command Portal is the part of SZL Holdings that's hardest to explain in a sentence, and the part that matters most.

Here's the sentence I use with investors: *Command Portal is the single cockpit a multi-domain operator — someone running defense intelligence, maritime, real estate, and advisory lines of business — uses to run all of them from one surface.* It is the product that makes the portfolio feel like a portfolio instead of a collection of apps.

Here's what it actually does.

The problem

When you run more than one vertical command platform, you immediately hit three issues:

1. **Context switching.** Every platform has its own login, its own navigation, its own alerts. An operator running Aegis, Vessels, and Terra is context-switching all day.
2. **Cross-domain signals.** A sanctions event flagged in Vessels might be a Prism Counsel concern and an Aegis tasking. A zoning change surfaced in Terra might affect a Carlota Jo advisory engagement. The domain platforms don't know about each other.
3. **Executive view.** The person running the whole operation wants a single surface that shows *what's happening across everything* without drowning in dashboards.

Command Portal solves all three.

The shape

Three layers:

1. **Unified navigation.** One authenticated session, one sidebar, one search. Domain modules (Aegis, Vessels, Terra, Carlota Jo, Prism Counsel, Forge) appear as surfaces inside the portal, not as separate apps. Role-based access controls what any given user sees.
2. **Cross-domain signal bus.** Every domain platform emits structured events — an Aegis indicator, a Vessels anomaly, a Terra pipeline event, a Prism Counsel compliance flag. The portal's signal bus routes those events to the right people across domains, with configurable rules.
3. **Executive command view.** The top-level surface for the operator running the whole thing. Cross-domain KPIs, a unified alert queue, and an AI-summarized daily brief. If you have twelve minutes in the morning to understand what happened across six domains, this is the surface that makes those twelve minutes enough.

Why it's hard

Three reasons, in order of difficulty:

Identity. One identity, many roles, across sensitive domains with different compliance regimes. A Vessels chartering user should not see the same things as an Aegis analyst, even in the "unified" portal. The role model has to be sharper than in a single-domain product, not looser.

Schema coordination. Cross-domain signals require the domain platforms to emit events in a shared envelope. Getting six product teams to agree on event shape is an ongoing organizational exercise.

UI coherence without monoculture. Each domain has its own operator culture. Maritime users have different expectations than defense users. The portal has to feel coherent without flattening the domains into one beige UI.

The AI layer

The portal is where the cross-domain AI features live. Three big ones:

- **Unified daily brief.** Every morning, a multi-source summary of the top signals across all domains the user has access to, with links into the relevant domain workspace.
- **Cross-domain pattern detection.** Signals that only make sense when you see them from two or more domains. The classic example: a Vessels sanctions flag plus an Aegis indicator plus a Prism Counsel counterparty alert — each benign alone, meaningful together.
- **Portfolio-level agents.** Agentic workflows that span domains — e.g., an onboarding agent that runs a new client through Prism Counsel compliance, creates the Forge project, provisions the domain access, and schedules the kickoff.

Every AI feature in the portal is logged, reviewable, and constrained by the same governance spine that sits underneath every domain platform.

Why this exists at all

You could argue the portal is a nice-to-have — six separate products would still work. Two reasons it's not.

One, it's the enterprise sale. An operator with three domain needs is nine times more valuable than an operator with one. The portal is what turns a product-by-product sales motion into a platform sales motion. Bloomberg didn't win by selling news, data, and chat as separate products. The unified surface is the product.

Two, it's how AI gets interesting. Single-domain AI is powerful. Cross-domain AI — where the signal in one domain informs action in another — is where the next generation of operator value shows up. The portal is the surface that makes that possible.

The state of play

The portal is shipped and running in internal use across the portfolio. External pilots are starting with two multi-domain operator groups — one focused on maritime + compliance, one on real estate + advisory + legal. I'll write about what I learn from both.

Next month I'll do a full portfolio deep-dive on Prism Counsel, which is the legal and compliance platform that sits underneath every domain and does more quiet work than any other product in the group.

Next post: how I hire, and why the first five hires in a multi-product group are completely different from the first five in a single product.

— S

SUBSTACK TAGS: COMMAND-PORTAL PLATFORMS PRODUCT

MEDIUM TAGS (PICK 5): PRODUCT DESIGN ENTERPRISE SOFTWARE ARTIFICIAL INTELLIGENCE

UX TECHNOLOGY

The First Five Hires in a Multi-Product Group

Why the standard early-stage hiring playbook breaks when you're running more than one product at once.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

The early-stage hiring playbook — hire two generalist engineers, a designer, a founding salesperson, and a chief of staff — is one of the best pieces of startup advice ever written. It is also actively wrong for what I'm doing.

When you're running a holding group with multiple command platforms from day one, the physics change. Here's the playbook that actually works, learned the expensive way.

Hire 1 — the platform engineer, not the product engineer

The single most important early hire in a multi-product group is someone who is obsessed with **shared infrastructure**. Not because they're the smartest person in the room — because the platform you share is the thing that determines whether three products feel like three products or three hundred.

What this person owns: the monorepo shape, the auth system, the data model conventions, the eval harness, the deploy pipeline, the telemetry, the API surface, the shared UI system. They don't build features. They build the substrate features are built on.

If you hire two product engineers before you hire this person, you will ship three products that share nothing and you will spend year two ripping them apart to share something.

Hire 2 — the vertical operator, not the salesperson

In a single-product company, hire two is often the first salesperson. In a multi-product group, hire two should be someone who **has actually operated in one of your verticals**. Ex-defense analyst for Aegis. Ex-chartering desk for Vessels. Ex-asset manager for Terra.

Their job is not to close deals. Their job is to make sure the first version of the product in *their* vertical is actually what operators in that vertical need. Without them, you build a product that looks right and feels wrong, and you don't find out until deal nine.

Sales will come. Domain truth has to come first.

Hire 3 — the design-engineering lead

Command platforms live or die on the quality of the surface. Not the cosmetics — the *surface*. How dense can the dashboard be without being overwhelming. How quickly can an operator move from signal to action. How confidently do they recover when the system makes a mistake.

This is not a visual designer. It's not a product manager either. It's a design engineer — someone who can hold the whole surface in their head, prototype in code, and ship production-quality UI without a

three-person handoff.

Every great command platform in history has had one of these people. Most fail because they don't.

Hire 4 — the second vertical operator

At this point you have platform, first vertical, and surface. Hire four is the operator for the second vertical. Same pattern as hire two. The reason to do this before a second product engineer: domain truth scales worse than code.

If you can only afford one of the two, always hire the domain person.

Hire 5 — the governance person

In a multi-product group selling into regulated verticals, there is a role that combines security, compliance, and a bit of operations. Role-based access, audit logs, SOC 2 runway, export control, data retention, incident response, customer legal reviews.

Most founders defer this to year two. The ones building for defense, maritime, real estate, and legal cannot defer it past hire five. Your first seven-figure deal will die on this desk if you don't have someone here.

This person will also write the playbook for the next ten hires across engineering, sales, and customer success — because governance is upstream of everything in regulated B2B.

What I deliberately did not hire

In year one I did not hire:

- A chief of staff. I wrote my own weekly operating rhythm. When it breaks, I'll hire one.
- A VP of engineering. Too early. The platform engineer leads engineering until we have two product teams.
- A head of marketing. The founder is the head of marketing in year one. This newsletter is part of that job.
- A CFO. A good part-time fractional does this better than a full-time hire until you're north of \$10M ARR.

The meta-lesson

The early team in a multi-product group is a **platform + vertical + surface** triangle. Everything else comes after those three points are drawn. Founders who try to hire the single-product playbook onto a multi-product reality end up with six engineers, no domain truth, and a UI nobody can use.

I'll write more team-building posts as the group scales. Next one I'm queueing: the operating rhythm — how I keep six products shipping with a small team without burning anyone out.

Next post: evals part two — what I actually look for in a production regression suite.

— S

What we got right

Three calls, in retrospect, that compounded.

Hiring the founding engineer before the first paid customer. Conventional advice says hire after revenue. We hired before, because the product wedge required someone who could reason about evals and guardrails from day one, and that person doesn't show up on a job board after you have customers — they pick the team that's already credible.

Hiring the customer-success person from a customer. The third hire came out of one of our pilot accounts. She'd spent three years inside a buyer that looked exactly like our next ten buyers. She knew the procurement language, the InfoSec playbook, and the political map of a typical buying committee. The pattern matching she brought in week one would have taken any external hire six months to develop.

Writing the operating cadence before the first hire. Weekly company-wide review (Monday, 45 min). Twice-weekly deep-work blocks with no meetings (Wednesday and Friday mornings). Quarterly off-sites that produce a written 3-page memo, not slides. We codified this before anyone joined, which meant nobody had to wonder how to behave. Culture by document beats culture by drift.

What I'd tell a founder hiring their first five

Hire the people who can do the next six months of work, not the next five years. The team you need at \$1M ARR is not the team you need at \$20M ARR. The hire who can scale across both stages is rare and expensive, and you usually don't need them yet. Hire for the current phase and renegotiate together when the company changes underneath you.

Pay above your stage. Equity is generous, salary is competitive but not extravagant. The first five take more risk than every subsequent hire combined and should be compensated for it. Don't use a "below-market salary, above-market equity" pitch to under-pay people who could be earning twice as much somewhere stable.

And: hire people who would still be your friends if the company failed. Most early-stage companies don't make it. The people who stay your friends regardless are the people you can be honest with when the numbers slip, and that honesty is the only thing that actually saves the company.

SUBSTACK TAGS:

HIRING

FOUNDER

TEAM

MEDIUM TAGS (PICK 5):

HIRING

STARTUP

LEADERSHIP

TEAM BUILDING

ENTREPRENEURSHIP

What Actually Breaks in Production AI

Ten months of incidents, categorized. The real failure modes nobody puts in the keynote.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

I keep a running log of every AI-related production incident across the SZL portfolio. Ten months in, here's the distribution. It does not look like what the frontier-AI conference talks tell you.

The rough breakdown of incidents by root cause:

1. **Schema drift** — 28%
2. **Retrieval regressions** — 22%
3. **Prompt drift / model updates** — 17%
4. **Rate limits and cost spikes** — 13%
5. **Data quality upstream** — 10%
6. **Model output quality degradation** — 6%
7. **Security / leakage** — 4%

"Model got dumber" is the *sixth* most common cause. Most of what breaks in production AI has very little to do with the model and a lot to do with everything around it.

1. Schema drift (28%)

By far the most common incident: the LLM returns JSON that doesn't match the schema the downstream code expects. A new optional field, a type that's now a string instead of a number, a date format that silently flipped from ISO to human-readable.

Fixes that work:

- Strict schema validation on every response. Never trust a model output structurally.
- Retry-with-repair on schema failure, with a bounded retry count.
- Log every schema failure to a stream. The trend tells you when a model update is coming before the vendor announces it.

2. Retrieval regressions (22%)

Second most common: the retrieval layer returned the wrong chunks, or the right chunks in the wrong order, and the model answered confidently from bad context.

This almost never looks like a failure at the model level. It looks like the model "hallucinating." It isn't. It's answering correctly from bad input.

Fixes that work:

- Retrieval-level evals as a first-class axis of the regression suite. Measure recall@k and MRR separately from end-to-end accuracy.
- Citation-required outputs. If the model can't cite a retrieved chunk, the output is suppressed.
- Periodic re-embedding when the embedding model updates. This is a silent killer.

3. Prompt drift / model updates (17%)

The vendor ships a new model version. Behavior changes subtly. Your carefully tuned prompt now produces different output.

Fixes that work:

- Pin model versions explicitly. Never let "auto" route to latest in production.
- Run the full regression suite on every candidate version before promoting.
- Keep the old version available as a rollback target for 90 days past promotion.

4. Rate limits and cost spikes (13%)

Traffic burst, a loop that should have bounded but didn't, a retry storm on a 429. Your bill 3x's in a day, or your latency explodes.

Fixes that work:

- Hard per-tenant and per-feature budgets. Kill switches, not warnings.
- Exponential backoff with a circuit breaker on persistent failures.
- Daily cost dashboards with per-feature attribution. If you can't tell which feature cost you \$40k last month, you don't have observability.

5. Data quality upstream (10%)

Garbage in, garbage out. The most entertaining incident in this category was a vendor silently changing a PDF template, which broke extraction in a way that only showed up as "slightly weird" numbers in downstream dashboards for two weeks.

Fixes that work:

- Input validation as aggressive as output validation. Shape, size, type, encoding.
- Canary inputs — a set of known-shape documents you process every hour as a health check.
- Anomaly detection on output distribution. If the mean extracted value shifts 20% in a day, page.

6. Model output quality degradation (6%)

The actual "model got worse" case. Rarer than you'd think when you have the other six cases under control.

Fixes that work:

- Gold-labeled regression suite, run nightly.
- Threshold-based alerting. Don't page on a 1% move. Do page on a 5% move.

- A/B on any real degradation — you may be on an unlucky sample, not a real regression.

7. Security / leakage (4%)

The lowest-frequency, highest-severity category. Prompt injection, cross-tenant leakage, inadvertent PII in logs, model memorization of sensitive training data.

Fixes that work:

- Never put raw user input directly into a system prompt that has tool access.
- Per-tenant isolation at every layer — retrieval index, cache, log stream.
- Red-team the system monthly with domain-specific attack prompts.
- PII scrubbing on all logs, turned on by default, never off.

The meta-lesson

Production AI is **90% plumbing, 10% models**. Founders who spend all their time on model selection and prompt tuning while ignoring the plumbing ship systems that demo well and fail live. The leverage is not in the magic. The leverage is in the boring parts.

If you're building anywhere in this stack, the single best investment you can make this quarter is a detailed incident log with root-cause categorization. Six months from now you'll know exactly where to harden.

Next post: inside Prism Counsel — the legal and compliance platform.

— S

SUBSTACK TAGS:

AI

RELIABILITY

ENGINEERING

MEDIUM TAGS (PICK 5):

ARTIFICIAL INTELLIGENCE

SOFTWARE ENGINEERING

DEVOPS

MACHINE LEARNING

RELIABILITY

Inside Prism Counsel: The Quiet Backbone

Legal and compliance tooling that doesn't look glamorous and moves more revenue than anything else in the portfolio.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

Prisim Counsel is the least-glamorous product in the SZL portfolio and the one that most often closes deals for the others. This post is about why.

What it is

Prism Counsel is the legal and compliance command platform that sits underneath every domain product. Five surfaces:

1. **Contract intelligence** — ingest, extract, and surface every obligation in a contract portfolio. Who owes what to whom, when, and under what conditions.
2. **Compliance runtime** — continuous monitoring against frameworks (SOC 2, ISO 27001, maritime sanctions regimes, defense export controls, real-estate regulatory filings).
3. **Matter management** — the thing a GC or legal ops lead runs their week on. Matters, deadlines, reviewers, status.
4. **Policy and playbook** — internal policies as first-class objects, versioned, reviewable, enforceable.
5. **Recovery tooling** — incident, breach, and dispute workflows with the evidence trail pre-assembled.

AI is inside every surface, doing extraction, drafting, redlining, summarization, and anomaly detection. The human reviewer always signs.

Why it's the quiet backbone

Three reasons.

One: enterprise deals die on legal. Every deal of size in defense, maritime, or real estate passes through a legal review. Counterparties have different risk appetites, different paper, different red lines. Prism Counsel is how the SZL portfolio ships deals through counterparty legal in hours instead of weeks. Internally, it's probably the single highest-leverage tool we run.

Two: compliance is the customer's trust question. When a defense buyer asks "what happens if your agent gets it wrong," the answer is not a feature. The answer is a process, an audit trail, a review queue, and a compliance posture. That is what Prism Counsel surfaces.

Three: it's cross-domain. A sanctions event flagged in Vessels is a Prism Counsel matter. A tenant complaint in Terra is a Prism Counsel matter. A breach detected in Aegis is a Prism Counsel matter. The platform is where cross-domain risk becomes a managed process.

What I've learned building it

Legal tech has more snake oil than any other AI-adjacent category. The honest lessons:

- **Contract extraction is 95% solved, 5% the thing that matters.** The 5% — the weird clauses, the amendments-to-amendments, the non-standard definitions — is where expert review is non-negotiable. The product discipline is to nail the 95% invisibly and put the 5% in front of a lawyer efficiently.
- **Playbook is a UI feature, not an AI feature.** The magic in legal ops is capturing the firm's own precedent and making it retrievable. AI helps retrieve and suggest. The precedent itself is the product.
- **"Agentic legal" is dangerous without scaffolding.** Any agent that touches a matter without a human reviewer in the loop is a liability. Every agentic feature in Prism Counsel has an approval queue by default.
- **The fastest path to value is not drafting.** It's intake — turning an email, a ticket, or a contract landing in inbox into a structured matter with the right reviewer assigned in seconds. Everything else compounds from there.

Who buys it

Today, Prism Counsel is deployed internally and in one external beta with a mid-market real estate group that runs 800+ contracts across 60 entities. The external buyer persona is a head of legal ops or a GC at a \$100M–\$2B revenue company with too much paper and too little tooling.

Over the next two quarters I'm going to write up a series on what legal ops actually looks like at that scale, because there's almost nothing useful published about it.

The pitch to builders

If you're building in legal tech, the advice I'd give: stop pitching "ChatGPT for lawyers." The buyer doesn't want that. The buyer wants *fewer surprises* — fewer things missed, fewer deadlines blown, fewer deals dying on redlines. Build the surface that reduces surprise. The AI is the ingredient.

Next post: what I look for in the first term sheet — and what I walk away from.

— S

How Prism Counsel is wired

Three layers, top to bottom.

The contract surface. Every contract enters Prism through one of four intake paths — an inbound NDA from a counterparty, a contract template our legal ops team is preparing for outbound, a renewal coming due, or an amendment to an existing agreement. Each path has a different default workflow, but they converge on the same review queue with the same red-line rubric and the same sign-off ladder.

The redline assistant. This is where the AI lives. Given a contract and our playbook (representations and warranties we always require, indemnity caps we accept, governing-law preferences, data-processing terms), the assistant produces a first-pass redline with explanations attached to every change. The lawyer reviews and edits, then accepts or rejects each suggestion. Over time the playbook learns from accept/reject patterns. Crucially: the AI never sends anything. A human always reviews and sends.

The audit ledger. Every contract, every redline, every approval, every counter-redline is logged with the user, the timestamp, the version diff, and the reasoning. When regulators or auditors ask "why did you accept this clause," there's an answer with a name on it.

Why this matters for the SZL portfolio

Every product in the SZL portfolio touches contracts. Aegis has accreditation paperwork and DoD subcontractor agreements. Vessels has charter parties, bills of lading, and broker agreements. Terra has joint-venture agreements, leases, and acquisition documents. Carlota Jo has master service agreements with every advisory client. Without Prism, contract overhead grows linearly with revenue. With Prism, it grows sub-linearly, because the playbook learns and the queue stays managed.

That's the bet. Not "AI replaces lawyers" — that bet ages badly. The bet is that legal ops becomes a force multiplier inside a multi-product company, and that the company that builds the legal-ops surface for itself can sell that surface to other multi-product companies that face the same problem.

There are a hundred AI-legal startups chasing this market. Most are building pure-play legal-AI products for law firms. Prism is building for in-house counsel inside operating companies, which is a different surface, a different buyer, and a different sales motion. We think the in-house surface wins, because that's where the volume of routine contract work actually lives.

SUBSTACK TAGS:

PRISM-COUNSEL

LEGAL-TECH

COMPLIANCE

MEDIUM TAGS (PICK 5):

LEGAL TECH

COMPLIANCE

ARTIFICIAL INTELLIGENCE

ENTERPRISE SOFTWARE

LEGALTECH

What I Look for in a Term Sheet

The parts of the deal that matter most, the ones that look fine and aren't, and the ones I walk from.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

I've now sat on both sides of a term sheet enough times to know what matters and what doesn't. Here is the short, honest list — the parts I optimize for, the parts that look fine but aren't, and the structural issues that make me walk regardless of valuation.

The three things I optimize for

- 1. Partner fit.** The single highest-leverage variable in any round. Not "fund fit" — *partner fit*. Who specifically will be in my board meetings. How they think about multi-product groups. How they react when a product misses. Whether they can make introductions that move the needle. I will take a flat round with a great partner over a 30%-higher round with a mediocre one, every time.
- 2. Information rights, not control.** Good investors want good information. Great investors want exceptional information and then trust you with decisions. I design for quarterly reporting rhythms that are detailed, candid, and predictable. That earns me latitude in the decisions between reports. Term sheets that substitute control rights for information discipline are red flags.
- 3. Pro-rata and follow-on signal.** The single most useful question in a first pitch: "if we hit plan, what's your follow-on posture?" The answer — specifically and numerically — tells me whether this investor is a one-shot transaction or a multi-round partner. Walk from vague answers.

The parts that look fine and aren't

Liquidation preference multiples above 1x non-participating. Looks small on a first round. Compounds catastrophically through three more. Non-negotiable: 1x non-participating.

Founder vesting restarts. Common with a new lead, especially at a premium valuation. Can be reasonable for *additional* equity, never for already-earned equity. Read carefully.

Drag-along below a board majority. A drag-along that can be triggered without a board majority and the common vote is a control mechanism wearing a procedural mask. Push back.

Board observer rights without definition. "Observer" can mean "silent guest" or it can mean "active voice with no fiduciary duty and no NDA." Define the scope. Require an NDA. Time-box the observer seat.

Participation rights on every future round, without cap. Fine in principle, dangerous in practice if it blocks strategic investors later. Standard is 2x rounds, not infinity.

The structural issues that make me walk

A lead that won't sign a clean term sheet without a side letter. Side letters are either needed or they're not. If they're needed, they should be visible to the cap table. Hidden terms are a cultural signal.

Unclear path to the next round. I ask every lead: "what would need to be true for you to lead the next round, and what would need to be true for you to pass?" If they can't answer the second part honestly, they can't be trusted with the first.

A lead that doesn't do reference calls as a two-way street. I will happily sit on a dozen reference calls. I also expect to call three founders the partner has backed through a down quarter. Any hesitation on the second is a walk.

Pressure to accept in under 72 hours. There is one exception: when the lead is responding to a signed term sheet from a competing fund. Outside that, urgency is a tactic, not a reality.

What I'd rather not over-optimize

Valuation by itself. A round raised at too high a valuation is a tax on the next round and a constant narrative problem. I'd rather close a round at 85% of the highest bid with the right partner and the right structure than 100% with either variable wrong.

Round size. Raise what you need for 18–24 months of plan plus a buffer. More than that creates its own problems — spending discipline decays, the team grows faster than the org can absorb, the next round bar is higher than necessary.

The meta-lesson

A term sheet is a contract, a signal, and a relationship in one document. The contract matters. The signal matters more. The relationship matters most. Founders who read term sheets as only the first are the ones who end up in bad marriages at good valuations.

I'll write more fundraising posts as the year goes on. If you're a founder approaching a raise and want to compare notes — especially if you're raising for a multi-product group — reply. This is a conversation that gets much shorter the second time you have it.

Next post: Vessels port-call intelligence, the deep technical dive.

— S

The post-money trap

The most common mistake first-time founders make is over-indexing on valuation and under-indexing on terms. A \$20M post on dirty terms is worse than a \$14M post on clean terms, and it isn't close.

Liquidation preference. 1x non-participating is the standard. Anything more — 1.5x, 2x, participating — means the founders and common shareholders take a worse outcome on every exit scenario below the home run. If a fund insists on participating preferred at this stage, that fund has told you something about how they think about the deal. Believe them.

Anti-dilution. Broad-based weighted average is the standard. Full-ratchet means that if you raise the next round at a lower price, the previous investors are made whole at the founders' expense. In a normal market this rarely matters; in the market we're actually in, it matters often.

Pro-rata rights. Reasonable for the lead. Unreasonable for everyone in the round. If everyone gets pro-rata on a \$5M round, you've sold the next round before you've raised it.

Information rights. Standard. The fund gets quarterly financials, an annual budget, and the right to inspect books with reasonable notice. Anything more — board observer rights for non-leads, monthly KPIs distributed to the entire syndicate — is overhead disguised as governance.

The terms that actually matter long-term

Two clauses do more work than the rest combined.

Founder vesting acceleration on change of control. Single-trigger means you vest fully on a sale. Double-trigger means you vest fully on a sale only if you're terminated without cause within twelve months. Double-trigger is the market norm and is reasonable. Single-trigger is rare and the wrong fight to pick.

Protective provisions. These are the things the company can't do without preferred-shareholder consent — taking debt above a threshold, selling the company below a threshold, changing the size of the option pool. The list should be short. If it's a page long, the fund is buying control in disguise.

The shortest version of all of this: read every line, ask out loud what it does in the worst case, and walk away from the round if the answer is "we lose the company." Nothing about this round is worth losing the company over.

SUBSTACK TAGS:

FOUNDER

FUNDRAISING

TERM-SHEET

MEDIUM TAGS (PICK 5):

FUNDRAISING

VENTURE CAPITAL

STARTUP

ENTREPRENEURSHIP

BUSINESS STRATEGY

How We Rebuilt Port-Call Intelligence From the PDF Up

The single most painful workflow in maritime operations, and the one that pays back the fastest when you fix it.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

If you want to understand why maritime is a software desert, sit with an operator during a port call.

The vessel arrives. The agent sends a Notice of Readiness by email. A few PDAs follow, all formatted differently. The Statement of Facts — the document of record — arrives sometime between 12 hours and 30 days after departure, as a scanned PDF, with handwritten annotations, in a layout that's unique to that agent in that port. Demurrage and despatch calculations happen in a spreadsheet model that someone built fifteen years ago. Disputes get resolved over email threads that eventually get archived into PDFs for the file.

Every port call produces five to twelve document artifacts, touches four to seven internal systems, and costs the operator somewhere between 90 minutes and six hours of back-office time.

Vessels rebuilt this workflow from the PDF up. Here's how.

The artifacts

The document types that flow through a port call are surprisingly stable across operators. The problem isn't variety — it's that each one comes from a different sender with a slightly different template:

- **Pre-arrival** — Notice of Readiness, port agent appointment, vessel particulars confirmation.
- **Arrival** — Statement of Facts in progress, tug and pilot movements, cargo readiness.
- **Operations** — loading/discharge tallies, bunker deliveries, stores delivery.
- **Departure** — final Statement of Facts, proforma disbursement account, final DA.
- **Post-call** — demurrage/despatch claims, dispute threads, agent performance.

The Vessels port-call intelligence module handles all five phases on one surface.

Stage 1: ingestion

Every document coming into the system — whether via email forwarding, agent portal, or direct integration — runs through a multi-stage extraction pipeline:

1. **Classification.** What document type is this, from which agent, for which voyage.
2. **Layout analysis.** Fixed-template fields (dates, times, tonnages) come out of typed PDFs cleanly. Scanned handwritten annotations go through a dedicated OCR path with confidence scoring.
3. **Entity extraction.** Structured events get extracted — "pilot on board at 14:30 local," "commenced loading at 15:12," "completed loading at 02:45," with a timestamp, a timezone, and a confidence level.

4. **Validation.** Extractions get cross-checked against the vessel's AIS record, the scheduled call, and the voyage plan. Anything that disagrees gets flagged.

The whole pipeline is eval'd against a gold-labeled set of port calls from eleven different ports and twenty different agents. Every model update runs the eval before promotion.

Stage 2: the structured port call

Once the documents are extracted, the port call becomes a single structured object. Every fact has a source, a timestamp, and a confidence. Operators see a timeline view of the call, with documents pinned at the relevant events.

Disputes — a port call's most expensive side effect — become tractable at this layer. Instead of hunting through email chains, the operator sees the event in question, every document that supports or contradicts it, and the chain of custody on the data.

Stage 3: demurrage and despatch

Demurrage and despatch calculations are the top ROI use case. A single disputed port call can involve six figures of claim exposure. Most operators run these calculations in spreadsheets built by people who retired.

Vessels runs them inline, with the charter party terms applied automatically, and produces a claim-ready output with evidence attached. Calculations that used to take half a day take ninety seconds. Claims that used to sit in inboxes for months ship the day after the call.

Stage 4: agent performance benchmarking

Because every port call is now structured data, operators can finally answer questions that have been unanswerable for a generation: which agents are fastest through loading? Which ports have the worst delay variance? Which counterparties consistently underreport laytime?

This is the surface that turns port-call intelligence from a cost-reduction play into a commercial one. Better agent selection. Better port selection. Better counterparty negotiations. Real data in rooms that have run on instincts.

What went wrong along the way

Three mistakes, in order:

One: we tried to build a "universal" extraction prompt that handled every document type in one pass. It worked in demos and failed in production. Rebuilt as per-document-type extractors with shared validators. Accuracy went up 22 points on the eval.

Two: we underestimated how much of the value was in the *timeline* rather than the individual extraction. Operators didn't care that we got the loading-start timestamp right; they cared that they could see the loading-start timestamp in context with the pilot-on-board, the weather hold, and the cargo-readiness event. Reshipped the surface around the timeline.

Three: we tried to automate dispute resolution. Bad idea. Disputes are a human negotiation with legal implications. The product discipline is to *prepare* the dispute packet — full evidence, calculations,

counterparty history — and hand it to a human. Agents who draft disputes without a human are a lawsuit with a UI.

What's next

The port-call module is live with three beta operators across tanker, dry bulk, and container fleets. The next module I'm queued up to write about is voyage P&L with live freight rate benchmarking — because once you have structured port calls, the voyage P&L finally becomes real-time.

If you run port calls and want to see the walkthrough, reply.

Next post: the founder's weekly operating rhythm, in detail.

— S

SUBSTACK TAGS:

MARITIME

VESSELS

PORT-CALLS

MEDIUM TAGS (PICK 5):

MARITIME

LOGISTICS

ARTIFICIAL INTELLIGENCE

SUPPLY CHAIN

DATA ENGINEERING

My Weekly Operating Rhythm, In Detail

Six products, a small team, and a calendar that has to not eat itself.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

People keep asking how I run six products with a small team and not lose my mind. The honest answer is: I don't always succeed, but I have a weekly rhythm that works more often than not. Here it is in full. Steal what's useful.

The week-level shape

- **Monday** — direction. The whole week's priorities are decided before Monday afternoon. Not reviewed — *decided*. Every product has one thing that matters most this week.
- **Tuesday / Thursday** — deep work. No meetings before noon, no exceptions. Writing, product review, architecture calls, coding if needed.
- **Wednesday** — external. Customer calls, investor calls, partner calls. Stacked.
- **Friday** — review and write. The week's output across products reviewed. This newsletter drafted.
- **Saturday** — off. Mostly.
- **Sunday** — planning. Next week's Monday prep. 60 minutes.

The anchors are the deep-work days and the planning hour. Everything else flexes.

The daily shape

Inside each day, a consistent structure:

- **06:30** — up, first coffee, no screen for the first 30 minutes.
- **07:00** — the morning brief. Command Portal, twelve minutes. What happened across every product overnight. Alerts triaged. Top three things surfaced.
- **07:30** — one deep block, two hours. The single most important piece of output for the day lands here.
- **10:00** — standups if needed, no more than 15 minutes per product.
- **11:00** — second block, two hours. Continuation, or a different product.
- **13:00** — lunch, walk, no screens.
- **14:00** — meetings / external.
- **17:00** — reviews, async comms, asks and answers.
- **18:30** — close the day. Write tomorrow's top three down. Computer off.
- **Evenings** — reading, sometimes writing. No operational work unless there's an incident.

This is not aspirational. This is what actually runs, most weeks.

The monthly shape

Every month has four fixed events:

1. **Product review per product** — 60 minutes each, structured agenda (metrics, top wins, top risks, asks).
2. **Portfolio-level operating review** — 90 minutes, cross-product KPIs, resourcing decisions.
3. **Customer council** — curated group of customers across products, 60 minutes, agenda set by them.
4. **Investor update** — written, not a call. Detailed enough that investors don't need the call.

That's it. Four meetings that shape everything else.

The six rules I try to hold

1. **One product-changing decision per week, maximum.** More than that and execution shreds. Less and the portfolio stagnates.
2. **No meetings before noon two days a week.** Non-negotiable. Violated once, restored immediately.
3. **Every customer conversation generates at most one engineering ticket.** If a call generates four, the product has a coherence problem, not an execution problem. Diagnose that before executing.
4. **Write every week.** This newsletter is part of the operating system, not a marketing activity.
5. **Say no in the same week the ask arrives.** A no two weeks later is three weeks of other people waiting.
6. **Close the loop.** Every customer who raised an issue gets a follow-up, regardless of outcome. Every investor who asked a question gets an answer. Every team ask gets a decision, even if the decision is "not yet."

What I use

No magic stack. A calendar, a text file per product with the week's priorities, a command portal for cross-product view, and a running log of decisions with dates. The best tool in my stack is the discipline of writing yesterday's top three down last night and the week's top thing down last Sunday.

What breaks, and how I fix it

The most common failure mode: customer-facing week bleeds into deep-work week. I notice it when two Thursdays in a row have meetings before noon. The fix: decline the next week's first three meeting requests before noon without negotiation.

The second most common failure: too many product-changing decisions. I notice it when three products all have "big new direction" threads open at once. The fix: pick one, tell the others "next cycle," write it down.

Next post: how I think about defensibility in vertical AI.

— S

What the week buys me

The cadence is built to produce three outputs every week, on schedule, regardless of what's on fire.

One written investor-style update. Drafted Friday afternoon, ready to send Monday morning to the cap table or to publish in public on the newsletter. Forces me to put numbers next to narrative every seven days. The discipline of writing the update is more valuable than the update itself — the questions I can't answer in writing are the questions I have to chase down before next Friday.

Two long-form essays. Tuesday and Friday for the newsletter. Drafts written on Monday and Thursday respectively, edited the morning of publication. The writing is operations work, not marketing — it's the artifact that forces me to think clearly about whatever is in front of me, which is nearly always also what the team should be thinking clearly about.

One product decision per pillar, per week. Six pillars, six decisions. Some weeks a pillar gets nothing more than "stay the course" — that's still a decision and it gets logged. The discipline of forcing one explicit decision per pillar per week prevents the slow drift that kills multi-product companies.

What the week protects against

Multi-product companies fail in two characteristic ways. The first is one product starves the others — usually whichever product the founder loves most or which has the loudest customer. The second is that no product gets a clean shot because the founder context-switches every twenty minutes. The cadence above is designed to make both failures visible early.

The Monday review surfaces resourcing imbalances within the first week they happen. The deep-work blocks (Wednesday and Friday mornings, no meetings, no Slack) ensure that at least four hours a week per pillar gets real attention from the founder. The weekly investor-style update makes it impossible to coast.

The cost of the cadence is that it's rigid. The benefit of the cadence is that it's rigid. Founders who run on adrenaline and inbox triage cannot operate six product lines, and there's nothing clever about the rhythm that fixes that — the rhythm only works if you actually keep it.

SUBSTACK TAGS:

FOUNDER

OPERATIONS

PRODUCTIVITY

MEDIUM TAGS (PICK 5):

PRODUCTIVITY

LEADERSHIP

ENTREPRENEURSHIP

STARTUP

TIME MANAGEMENT

Defensibility in Vertical AI

Where the moat actually sits when the models commoditize.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

Everybody is writing about AI moats right now. Most of it is wrong.

The writing falls into two camps. One camp says there are no moats in AI because the models are commoditizing. The other camp says the moats are huge because data, distribution, and talent compound. Both camps are looking at the wrong layer.

In vertical AI, the moat is **neither the model nor the data in the abstract**. It's six specific things that have to be built together. Here's the list.

1. A proprietary workflow ontology

The vocabulary of the domain, in software. Berths, bunkers, port calls, laytime events in maritime. Parcels, overlays, comps, roll schedules in real estate. Targets, indicators, RFIs, collection plans in defense. Matters, clauses, playbook entries, obligations in legal.

This is the thing nobody can copy in a weekend. The ontology comes from thousands of hours with operators, refined through product use, and it's what makes your system answer the right questions instead of generic ones.

2. An eval set that nobody else can reproduce

Gold-labeled, domain-expert-curated, representative of your customers' real traffic. This is an asset that compounds. Every month you run it, it gets better. Every new customer feeds it. Competitors can't shortcut it.

3. Connected workflows that already earn trust

Once an operator's work passes through your system — port calls, IC memos, intel products, legal matters — the switching cost is not the data export. It's the trust the system has accumulated. The ritual it's part of. The dissent path. The review queue. Trust is expensive to build and cheap to maintain, which is the exact opposite of most moats.

4. Governance posture

Role-based access, audit logs, redaction, eval harness, review queues, SOC 2 or better, industry-specific regimes (ITAR, FedRAMP, relevant maritime regimes, relevant real-estate regulatory touchpoints). Regulated buyers cannot deploy a system without these, and assembling them takes years. New entrants underestimate this constantly.

5. A distribution pattern the horizontal vendors can't replicate

You know the ten investors, the forty buyers, the hundred builders who matter in your vertical. You're in their inbox every week. You speak at the three conferences they attend. You write the newsletter they forward. This is not a moat in the traditional sense — it's a distribution habit — but it compounds and it's expensive for a horizontal vendor to fake.

6. A portfolio effect

When you have more than one vertical command platform in the same group, each new one ships faster. The platform spine — auth, evals, guardrails, shared UI, governance — amortizes across products. New domains piggyback on proven infrastructure. The second product ships in one-third the time of the first.

This is the thing that makes SZL Holdings, specifically, a defensible bet: not any one of Aegis, Vessels, Terra, or Carlota Jo in isolation, but the group. The spine is the durable asset.

What is *not* a moat

Despite what you'll read:

- **Your prompt library.** Prompts leak, and the good ones get commoditized by the model itself.
- **Your fine-tune.** Useful, cheap, rarely a moat by itself. Next-gen base models usually match or exceed fine-tune-on-older-base performance.
- **Your data, by volume.** Volume without structure is nothing. Small, structured, labeled domain data is everything.
- **Your UI polish in isolation.** Polish is table stakes. Polish plus workflow is the product.
- **Brand, this early.** Brand becomes a moat at scale. Before scale, it's marketing expense.

The practical test

When I evaluate a vertical AI company — my own products, competitors, investments — I run the six-point test:

1. Do they have a proprietary ontology in software?
2. Do they have an eval set competitors can't replicate?
3. Do they have workflows that already earn operator trust?
4. Do they have the governance posture regulated buyers need?
5. Do they have a distribution habit in their vertical?
6. Do they have a portfolio effect (or are they a single-product shop, which is fine, just different)?

Companies scoring 5–6 are durable. Companies scoring 3–4 are acquisition targets. Companies scoring 0–2 are feature sets waiting to get eaten by the nearest horizontal vendor's next release.

This is not a complicated framework. It's the one I wish more founders and investors used.

Next post: the monetization ladder — free, paid, consulting, and when to open each rung.

The four moats that compound

In order of how hard they are to build, ascending.

Workflow integration. The product is wired into the customer's daily process. Switching costs are real because the customer has trained their team, written their SOPs, and integrated their other tools around the product. This moat is shallow on its own — competitors can copy the workflow — but it's the entry ticket. Without it, every other moat is theoretical.

Domain data flywheel. Every customer interaction makes the product better for the next customer in the same vertical. Vessels gets better at predicting port-call delays because every charter that runs through it adds another data point. The data isn't proprietary in the abstract sense — anyone could collect AIS — but the labeled, validated, in-context data is, and the lead compounds.

Trust infrastructure. Audit logs, evals, accreditation, certifications, references from operators the buyer respects. This moat takes years and cannot be bought. It also cannot be replicated by a horizontal AI vendor without becoming a vertical AI vendor, which is a different company.

Operator network effects. When operators inside a vertical use the product to coordinate with each other — a charterer and an owner using the same Vessels surface to negotiate a fixture, two analysts in different agencies using Aegis to share a fused intelligence picture — switching costs become collective. The operator can't switch unless their counterparties do too.

Why horizontal AI doesn't catch up here

Horizontal AI vendors compete on capability per dollar. That's a brutal race because the cost of capability is falling 60–80% per year. The vendor that wins this year loses next year if they don't rebuild.

Vertical AI competes on integrated workflow correctness in a specific domain. The cost of correctness in a regulated domain is dominated by trust-building, accreditation, and reference-customer cycles, none of which compress at Moore's-Law rates. The vertical product that's two years ahead on trust infrastructure stays two years ahead, because the buyer's procurement process literally cannot move faster than that.

This is why the next decade of enterprise software belongs to operator-grade command platforms with shared AI infrastructure underneath. The infrastructure compresses on horizontal-AI economics. The customer-facing surfaces compound on vertical-AI economics. We get the best of both.

SUBSTACK TAGS:

AI

STRATEGY

MOATS

MEDIUM TAGS (PICK 5):

ARTIFICIAL INTELLIGENCE

BUSINESS STRATEGY

SAAS

VENTURE CAPITAL

PRODUCT

The Monetization Ladder

Free, paid, consulting, enterprise — when to open each rung and when to keep it closed.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

Monetization in a multi-product group is not a single pricing decision. It's a ladder. Each rung serves a different customer and a different function. Open them in the wrong order and you leave money on the table or, worse, accidentally train your audience to expect the wrong thing.

Here's the ladder I use for SZL Holdings, and when I open each rung.

Rung 1 — Free content

The newsletter, the open-source repos, the public walkthroughs, the conference talks. Cost is real (my time) but not directly priced. Purpose: distribution, credibility, feedback loop.

Open from day one. Never close.

Rung 2 — Paid newsletter / community

Founding-member tier. Deeper portfolio deep-dives, quarterly investor-style memo, small monthly office-hours call. Priced low enough that it's a signal rather than a revenue center (\$15/month or \$150/year, with a founding-member tier at \$500/year).

Open 60–90 days after the newsletter launches, once the free tier has a clear shape and an engaged readership.

Purpose: qualify the audience, fund the writing time, create a forum for the people who matter.

Rung 3 — Self-serve product trials

Free tier of Terra's market intelligence, Vessels' port-call demo, Aegis's simulator. Limited data, limited use, but real product. Open to anyone who signs up.

Open when the product is genuinely demo-ready, not before. A bad self-serve trial is worse than no self-serve trial — you're training your audience to see the product as incomplete.

Purpose: qualified leads, real-world usage data, feedback without a sales cycle.

Rung 4 — Paid pilot / design partner

Paid pilot engagements with a small number of serious operators. Priced as a 3–6 month engagement (\$25K–\$150K depending on scope), with clearly defined success metrics and a path to production.

Open when you have 2–3 named design partners ready. Never run more than 3 at a time in the first year. Pilots eat your team's time — underestimating this is the most common mistake.

Purpose: revenue, real deployments, case studies, product roadmap grounded in real usage.

Rung 5 — Enterprise SaaS

The production product. Annual contracts, multi-seat, governance-layer included, SSO / SCIM / audit logs / SOC 2. Priced per-seat for Terra and Carlota Jo, per-fleet for Vessels, per-enclave for Aegis, per-entity for Prism Counsel.

Open when you have one design partner in production at scale. "At scale" means their operational work genuinely runs on the system.

Purpose: durable revenue, scale.

Rung 6 — Consulting / advisory

Strategic advisory on AI platforms, vertical deployments, and command surface design. Small number of engagements per year, priced for the seniority of the work.

Open selectively. Consulting revenue is high-margin but non-scalable. It should fund the mission, not define it. I cap it at 15% of revenue to avoid the gravitational pull.

Purpose: funding, relationships, access to problems that inform the product roadmap.

Rung 7 — Platform licensing

Licensing the SZL AI platform spine (governance, evals, guardrails, shared UI) to enterprise buyers building their own vertical command products in-house.

Open much later — year two or three at earliest. Platform licensing is a different product shape and a different sales motion. Opening it too early fragments the team.

Purpose: unlocks a different tier of enterprise customer and a different revenue shape.

The rules for opening

Four rules I use for any rung:

1. **Never open a rung before you can serve it well.** A bad paid pilot is more expensive than no paid pilot.
2. **Close a rung if it's distracting from a higher-priority rung.** Consulting is the most common candidate for temporary closure.
3. **Never let the bottom two rungs atrophy.** Founders who get excited about enterprise revenue often let the newsletter and open-source work decay. That is a long-term mistake. Top-of-funnel takes years to rebuild.
4. **Price the top two rungs to reflect the work, not the market.** If you underprice enterprise to win a logo, you'll spend the next three years reclassifying that customer as "reference customer" instead of "revenue."

Where SZL is today

Rungs 1 and 2: open, paid tier launching soon.

Rungs 3 and 4: open selectively per product — Vessels and Terra have paid pilot engagements active; Aegis has a simulator demo but no self-serve trial (by design, the buyer profile doesn't use them).

Rung 5: open for Terra and Vessels; Aegis and Prism Counsel in controlled early-access.

Rung 6: open but capped.

Rung 7: closed — year-two, earliest.

Next post: what cross-posting from Substack to Medium actually does for reach.

— S

The shape of the ladder

Six rungs, from free to enterprise, each one earning the next.

Rung 1 — Newsletter. Free. Top of the funnel for the entire portfolio. The reader who subscribes to SZL Command is one click from any product page. Conversion is patient — measured in months — but the unit cost is zero and the audience compounds.

Rung 2 — Founding-member tier on the newsletter. \$500/year, capped at 100 seats year one. Pays for the editorial cadence and produces a small private community of operators, investors, and buyers who self-select as serious. About a third of these become product leads or advisory leads within twelve months.

Rung 3 — Carlota Jo advisory engagements. \$100K–\$500K per engagement. Sold off the back of the newsletter and the founding-member community. Margins are excellent but the ceiling is the founder's calendar. Advisory is a wedge, not a destination — every engagement is qualified for whether it can become a product pilot.

Rung 4 — Product pilots. \$25K–\$100K for a 60–90 day paid pilot of one of the SZL platforms (Aegis, Vessels, Terra, etc.). Pilot scope is narrow on purpose — one workflow, one team, one measurable outcome. Conversion to ARR is the metric that matters; we target 60%.

Rung 5 — Product subscriptions. \$50K–\$500K ARR per platform per customer. The default contract is one-year with annual increases tied to a published index. Customer-success motion starts at signature — pilots that converted have a CS handoff inside the first week of the paid contract.

Rung 6 — Multi-product enterprise agreements. \$1M+ ARR. A single customer using two or more SZL platforms under one agreement, with cross-platform discounts and shared governance terms. These take 12–18 months to develop but are the long-term shape of the business.

What the ladder protects

It protects against two failure modes that kill multi-product companies. First, the discount-pilot trap — agreeing to a \$1 pilot in exchange for a logo and never converting it. Every rung above has a price floor and a stated conversion expectation. Second, the advisory-trap — letting consulting revenue substitute for product traction. Advisory is capped explicitly at 30% of revenue in year one and 15% by year three. If we miss those targets, we're a consulting firm with a software side project, which is not the company we're building.

SUBSTACK TAGS:

FOUNDER

MONETIZATION

PRICING

MEDIUM TAGS (PICK 5):

PRICING STRATEGY

SAAS

BUSINESS STRATEGY

ENTREPRENEURSHIP

REVENUE

Cross-Posting Strategy: Substack, Medium, LinkedIn, X

What each surface actually does, which posts belong where, and how the portfolio compounds.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

I publish the same way I ship product: one source of truth, multiple surfaces, each with its own job. Here's how the cross-posting works for this newsletter.

The surfaces and their jobs

Substack is the home base. Canonical URL, owned email list, paid tier, and the archive I send to investors and buyers when someone says "tell me more about what you're building."

Medium is the discovery engine. Medium's distribution is real — the tag system and the Partner Program put long-form in front of readers who would never find me otherwise. Cross-posting to Medium is cheap and the exposure compounds.

LinkedIn is the professional amplifier. A short post (150–250 words) that summarizes the main argument and links back to the Substack original. Best reach in the B2B audience I actually care about.

X is the signal flare. A short thread (6–10 posts) with the post's best ideas, ending in a link. Good for reach among the builder and investor audiences. Worse ROI than LinkedIn for enterprise-buyer audiences.

Each surface has a different audience, a different format, and a different pace. The content is the same. The framing is different.

The rhythm

Every post ships in four stages:

1. **Tuesday 09:00 ET** — Substack long-form goes live. Email to subscribers.
2. **Tuesday 09:10 ET** — LinkedIn short-form goes live with a single-image header and a link.
3. **Tuesday 09:15 ET** — X thread goes live. Link in the last post.
4. **Wednesday 09:00 ET** — Medium cross-post goes live with canonical URL pointing back to Substack.

The 24-hour delay on Medium protects Substack SEO and email-open stats. The canonical URL tells search engines which is the original.

The Medium tag discipline

Medium tags are a distribution lever most writers waste. Three rules:

1. **Five tags per post, always.** Use the full budget.
2. **Mix broad and narrow.** One big tag (Artificial Intelligence, Startup) for reach, three middle tags (Enterprise Software, SaaS, Product Design) for targeting, one narrow tag (Maritime, Defense Technology, Real Estate Investing) for the specific audience.
3. **Submit to a relevant Medium publication where possible.** Publications have their own distribution. Query the fit. "Towards AI," "The Startup," "Better Programming," "Data Driven Investor," and domain-specific pubs like "Inside Safety" for defense angles.

The LinkedIn discipline

LinkedIn rewards three things the other surfaces don't:

1. **Personal framing.** "I was wrong about X" and "here's what I learned" outperform "here's a framework" by a wide margin.
2. **Short, scannable paragraphs.** One-line paragraphs. Whitespace. The opposite of the long-form version.
3. **A clear CTA at the end.** "Full version on Substack →" with the link. Do not use the link as the headline — LinkedIn penalizes it.

The X discipline

X threads are harder than they look. The rules I use:

1. **Hook post is everything.** 60% of the read drops at post 1. Lead with the sharpest claim in the essay.
2. **One idea per post.** No paragraph breaks. No "continued."
3. **Thread length is 6–10.** Shorter loses the medium's thread algorithm; longer loses readers.
4. **Link only in the last post.** Links earlier in a thread crater reach.

What I don't do

- **Repost the full Substack body on LinkedIn.** LinkedIn Articles are where writing goes to die. Summary + link wins.
- **Post on X more than once per day for the newsletter.** Audience burns out.
- **Post the same content to Reddit.** Wrong format, wrong norms, usually ends badly unless you are already a regular contributor in the relevant subreddit.
- **Use scheduling tools that break the link previews.** Native posting wins on every surface.

What the numbers look like

My current baseline, for reference:

- Substack post → ~2,500 opens, ~900 clickthroughs to archive or external link.
- LinkedIn post → ~8,000 impressions, ~300 clickthroughs, ~20 meaningful comments.
- X thread → ~20,000 impressions, ~400 clickthroughs, ~30 meaningful replies.
- Medium cross-post → ~1,200 views week one, long-tail to 3–5,000 over 90 days via tag discovery.

Medium is the surface that compounds quietly. Substack is the surface where the relationship lives. LinkedIn is where the buyers are. X is where the builders are. Every post serves all four.

Next post: inside Forge — the client delivery platform.

— S

The mechanics, in order

Tuesday and Friday at 09:00 ET, an essay goes out on Substack to the email list. That's the canonical version. Everything that follows points back to it.

Twenty-four hours later, the same essay is published on Medium under the SZL Command publication, with the canonical URL set to the Substack permalink. Medium gets the Partner Program views and the SEO discovery surface; Substack keeps the email open rates and the subscriber relationship. The 24-hour delay matters: published simultaneously, Google sometimes ranks the Medium version higher for our own essay, which steals the new-subscriber funnel.

Same day as the Substack send, the founder accounts post a 150–250 word LinkedIn version with one branded image and a single CTA back to the Substack. LinkedIn rewards single-CTA posts with longer dwell time and replies — three CTAs cuts engagement by half.

Same day, an X thread of 6–10 posts — one idea per post, image only on post one, link only on post eight. Threads built this way get 3–5x the impressions of single posts with a link in them, because X's distribution model penalizes outbound links.

The amplifications that don't work

A few things I've stopped doing.

Comment-swap circles. Trading comments with five other founders to manufacture engagement. The platforms detect this within weeks and the cost is paid by the audiences, who learn to ignore the swap.

Scheduled identical cross-posts. Same image, same caption, same link, four platforms, queued in Buffer. The audience knows immediately. Each platform deserves a native version that respects that platform's rhythm.

Newsletter swaps with non-aligned audiences. "We have 10K subs, you have 10K subs, let's recommend each other." The conversion rate is dismal unless the audiences actually overlap. We do three swaps per quarter, all with publications whose audience could plausibly buy one of our products.

The honest answer is that compounding distribution comes from publishing twice a week, on schedule, for two years, with zero exceptions. There is no growth hack that substitutes for that. Everything in this essay is amplification of consistency, not a replacement for it.

SUBSTACK TAGS:

WRITING

DISTRIBUTION

CONTENT

MEDIUM TAGS (PICK 5):

CONTENT MARKETING

WRITING

SOCIAL MEDIA

PERSONAL BRANDING

MARKETING

Forge: Client Delivery as a Product

Professional services are a \$2T category. The delivery systems inside them are still built on SharePoint.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

Professional services — consulting, advisory, implementation, managed services — is a \$2 trillion global category. The delivery systems inside it are mostly built on SharePoint, Google Drive, and heroics.

Forge is the SZL platform for running client delivery as a product. This post is about why it exists and what it does.

The problem

Any firm that delivers engagements to clients — a consulting practice, an implementation team, a managed-services provider, an advisory firm — has the same pattern. Each engagement looks roughly like:

- **Kickoff**, where scope, team, timeline, and deliverables are agreed.
- **Delivery**, where the work happens.
- **Governance**, where status, risks, and changes are tracked.
- **Closeout**, where deliverables land and invoices go out.
- **Relationship**, where the next engagement is (or isn't) sold.

Every firm does these five things. Almost none of them runs them on a purpose-built system. The tooling is Google Drive for documents, Excel for resourcing, Jira or Asana for tasks, a CRM for the relationship, and a lot of email holding it together.

The cost: slippage, margin erosion, delivery quality variance, and partners who spend 30% of their time on admin instead of client work.

What Forge is

Forge is the delivery command platform for professional-services groups. Five surfaces:

1. **Engagement command** — every engagement as a structured object. Scope, team, deliverables, milestones, risks, status. The single source of truth.
2. **Resourcing** — who's on what, how much of their time, when they free up. Across the whole firm, not one partner's mental model.
3. **Client satisfaction** — structured feedback collection, NPS tracking, red-flag alerts. Real data, not anecdotes.
4. **Delivery library** — templates, playbooks, reusable assets. Knowledge that used to live in partners' heads, now first-class and retrievable.

5. **Commercial pipeline** — the relationship side, integrated with delivery so the same client doesn't get three people calling.

AI sits inside each: structured notes from client calls, auto-generated status reports, risk flagging from engagement signals, retrieval from the delivery library. Humans sign every client-facing artifact.

Why this fits the SZL portfolio

Two reasons.

One: Carlota Jo — SZL's advisory brand — is a consulting practice. Forge is the system Carlota Jo runs on. Dogfooding is a good reason but not the whole reason.

Two: every SZL product ships with a services component in the first year — implementation, customization, success management. Forge is how that services layer stays profitable rather than eating product margin. Consulting firms and in-house services teams are the external buyers.

The hard parts

Three I've learned so far.

Partner resistance. Partners do not love transparency about their own delivery quality. Forge's client-satisfaction surface makes delivery quality measurable across partners. The product discipline is to frame it as "protecting margin" rather than "measuring partners." Same feature, different narrative. Both are true.

Resourcing reality vs. the plan. Every firm plans resourcing as if people work eight billable hours a day. They don't. A realistic resourcing model bakes in a utilization target (65–80% for most firms), accounts for ramp, and flags sustained over-utilization as a risk, not a win.

The delivery library vs. confidentiality. Every asset in the library has a client origin. Reusing it on the next engagement without scrubbing confidentiality is a lawsuit. Forge has a dedicated sanitization workflow for any asset that crosses from one client to another, with a human reviewer in the loop.

The metric I watch

The single most predictive metric for a services firm's health, in my experience: **percentage of engagements ending with a signed case study and a referenceable client.** Not utilization. Not margin. Not NPS by itself. That composite metric says — we delivered, they're happy, and they'll say so.

Forge surfaces it automatically. For Carlota Jo, the number is a leading indicator of every other financial metric.

Who buys Forge

Two buyer profiles today:

1. **Boutique consulting firms** (5–50 partners) whose delivery tooling hasn't scaled with them.
2. **In-house services teams** at enterprise software vendors — the implementation and customer-success orgs — who need professional-services discipline without professional-services-firm process overhead.

I'm not selling to the Big Four. They'll build versions of this internally, badly, and ignore me. Fine.

Next post: the investor update, in public, Q1 edition.

— S

What Forge does, concretely

Three things.

Project ledger. Every paid engagement — pilot, advisory, full-product subscription — has a single ledger row. Scope, milestones, deliverables, payment schedule, current status, owner, customer-success contact. The ledger is the single source of truth, and it's queried by every other surface (Command Portal, monthly investor update, finance close).

Delivery playbooks. For each repeatable engagement type — Aegis pilot, Vessels production rollout, Carlota Jo strategy engagement, Terra underwriting integration — Forge holds the playbook: the kickoff agenda, the week-by-week plan, the artifacts produced at each milestone, the sign-off rubric. New team members onboard against the playbook, not against tribal knowledge.

Customer-facing portal. Every customer gets a single URL where they can see their project status, milestones hit, deliverables shared, invoices, and the next decision they owe us. This replaces the email/Slack/Drive sprawl that kills professional-services delivery for everyone who doesn't bring it under one roof.

Why we built it ourselves

Two reasons.

First, off-the-shelf PSA tools are built for consultancies that bill hours, and we don't bill hours — we bill outcomes against a fixed scope. The data model is wrong from the first screen. We tried two of the well-known tools and abandoned both within sixty days.

Second, Forge is the spine that lets us run multiple product lines and an advisory practice without losing fidelity. Every engagement that touches Carlota Jo, Aegis, Vessels, or Terra runs through the same Forge ledger, which means we have one number for delivery health across the whole portfolio. That number is the most reliable leading indicator of churn we have.

Forge is not a product we sell, at least not yet. It's an internal platform that may, in 18–24 months, become a product for the small number of multi-product companies that have the same problem. For now it's a moat — the operational reason we can run six product lines with a small team without dropping deliverables.

SUBSTACK TAGS:

FORGE

SERVICES

DELIVERY

MEDIUM TAGS (PICK 5):

CONSULTING

PROFESSIONAL SERVICES

PRODUCT MANAGEMENT

ENTERPRISE SOFTWARE

ARTIFICIAL INTELLIGENCE

Q1 Investor Update, In Public

The same numbers I send investors, published openly, with the good parts and the bad parts.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

One of the things I promised at the start of this newsletter: I'd publish the investor update in public. Here's Q1, in the same shape I send it to the cap table, with the numbers that are real marked as real and the ones I'm still stabilizing marked as such.

Headline

Q1 was a "shipping and qualifying" quarter. Three products moved from pilot-ready to pilot-in-flight. One product moved from pilot-in-flight to early production. Revenue grew from the low-six-figure annualized rate entering the quarter to the mid-six-figure annualized rate exiting. We are not yet at the rate where the business stops burning; we are at the rate where the burn is paying for demonstrable progress across every product.

Products

Aegis — three program offices in active pilot conversations; one allied partner scoped. No revenue yet; first paid pilot expected late Q2.

Vessels — three paid design-partner pilots active across tanker, dry bulk, and container. Port-call intelligence module live; voyage P&L module in early production with one partner.

Terra — two paid pilots with mid-market GPs. Underwriting workbench and market-intelligence surface in production. First LP-facing workflow shipped this quarter.

Carlota Jo — advisory engagements active with three enterprise clients. Forge (the delivery platform) dogfooded internally; first external Forge pilot scheduled Q2.

Command Portal — internal use across the portfolio; two external pilots starting.

Prism Counsel — internal use as legal-ops spine for SZL; one external beta with a real-estate group.

CORTEX, IMPERIUM, Autopilot — internal platform tooling, not yet a separate sales motion.

Financials

- Revenue, annualized run rate exiting Q1: **[mid-six figures]**, up from low-six entering.
- Gross margin: **72%** at the product level; lower when consulting revenue is blended in.
- Burn: **[on plan]**, approximately 65% of planned quarterly burn — reflects a later-than-planned second-vertical engineering hire.
- Runway: **[on plan]** months at current burn; materially longer if the Q2 paid pilot pipeline closes as expected.

Wins

- Three paid pilots closed in Q1, all on time, all with success metrics defined pre-contract.
- First production customer on Terra. Their team runs underwriting on the workbench full-time.
- Eval harness coverage expanded from 120 cases to 1,800 cases across the portfolio. Zero unflagged regressions in Q1.
- First full-portfolio customer advisory call went well — clear asks captured, roadmap adjusted.

Misses

- Second vertical engineering hire slipped by one quarter. I over-optimized for domain-fit on the hire; right call, cost us two weeks on the Vessels roadmap.
- One pilot customer churned after month two of a three-month engagement. Reason: a reorg on their side, not a product issue. Post-mortem captured; I'll write a full post on it.
- Command Portal's cross-domain alert bus shipped two weeks late due to an identity-model rebuild. The rebuild was the right call; the delay was real.
- Writing cadence on this newsletter slipped once in Q1 (missed a Friday). Won't happen again.

Roadmap through Q2

Three priorities per product — two existing, one new — and one portfolio priority.

- **Aegis:** disconnected enclave mode. OT/ICS ingest deepening. First paid pilot.
- **Vessels:** voyage P&L live with freight-rate benchmarking. Bunker optimization module in beta. Compliance command expansion (FuelEU Maritime).
- **Terra:** LP reporting workflow in production. Climate-risk model expansion. First multi-fund customer.
- **Carlota Jo / Forge:** first external Forge pilot. Delivery-library scale-up. Client-satisfaction surface benchmarking across engagements.
- **Command Portal:** external pilots in production. Unified daily brief shipped.
- **Prism Counsel:** first external production customer. Sanctions regime expansion.
- **Portfolio priority:** SOC 2 Type II attestation process kicks off this quarter.

Asks

If you're reading this:

- **Intros.** Maritime operators considering voyage-P&L tooling. Defense primes with modernization spend. Mid-market real-estate GPs with weak underwriting tooling. Heads of legal ops at \$100M–\$2B revenue companies.
- **Candidates.** Design engineer for the Command Portal surface. Second-vertical operator hire (maritime). Governance and compliance lead.
- **Partner referrals.** Warm intros to funds that back multi-product groups specifically.

Replies are read. The next investor update will be public in the same format after Q2.

What's working

Pipeline density. Inbound qualified conversations are up 4x over Q4. The newsletter and the design-partner motion are both contributing. The mix of sources is healthier — no single channel is more than 40% of qualified inbound.

Pilot conversion. Of the four pilots that ended in Q1, three converted to paid annual contracts. The fourth slipped to Q2 because of customer-side budget timing, not product fit. 75% conversion is above our model and the cohort is small enough that we should not extrapolate.

Team retention. Zero regrettable departures. Every offer extended in Q1 was accepted. Salary bands set in Q4 are holding.

What's not working

Sales cycle length. Average time from first qualified call to signed paid pilot is still 84 days. Target is 30. The gap is mostly in legal review, which we have started to address with the Prism Counsel template playbook, but the impact won't show until Q2.

Aegis accreditation timeline. The IL5 accreditation work is six weeks behind the plan we laid out in October. The slip is partially our pace and partially the government partner's review queue. Conservative estimate: cleared in late Q3 instead of late Q2.

Newsletter paid conversion. Free-to-paid is at 3.1%, against a target of 5–8% within sixty days. Two diagnoses underway: are we asking for the upgrade too late, and is the founding-member offer priced correctly? Expect a reset on this in early Q2.

What's next

In order of priority, the next ninety days.

1. Close two more enterprise pilots in defense and one in maritime.
2. Ship the Aegis v1.2 release with the new evals harness and the audit-trail rebuild.
3. Hire one senior engineer for Vessels and one customer-success lead for Terra.
4. Run the first founding-member office-hours call and use it to recalibrate paid-tier pricing.
5. Publish the next twelve newsletter essays on the existing schedule with no slippage.

The shape of Q2 is one quarter of execution against the Q1 thesis, with the next strategic decision deferred to a written memo at the end of June. We will not change the thesis on adrenaline.

SUBSTACK TAGS: [FOUNDER](#) [INVESTORS](#) [METRICS](#)

MEDIUM TAGS (PICK 5): [STARTUP](#) [VENTURE CAPITAL](#) [METRICS](#) [ENTREPRENEURSHIP](#)
[TRANSPARENCY](#)

How I Decide What Becomes an Agent

The decision tree for when a workflow should be a UI, a one-shot generation, or a fully agentic loop.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

Every team building production AI has this conversation: "should this be an agent?" It's usually asked too late, too vaguely, and with too much enthusiasm. Here's how I actually decide, with the tree I use.

The three shapes a workflow can take

A given workflow almost always fits one of three shapes:

1. **UI-first.** The operator drives. AI provides ingredients — suggestions, summaries, auto-fills — but the operator is in the seat.
2. **One-shot generation.** The operator provides inputs; the AI produces a structured output; the operator reviews and ships. No multi-step reasoning. No tool use.
3. **Agentic loop.** The AI plans, uses tools, observes results, and iterates toward a goal, potentially with a human reviewer at checkpoints.

These three shapes have wildly different cost profiles, reliability profiles, and product-design implications. Picking the wrong one is the single most common AI product mistake I see.

The decision tree

I ask four questions, in order, for any proposed workflow.

1. Is the output structure fixed and small?

If yes — if the goal is "produce this structured record" or "fill in these fields" — the answer is almost always one-shot generation or UI-first with AI assist. Not an agent.

If no — the output is open-ended, or the structure depends on what the agent discovers — continue.

2. Does it require more than one external action to complete?

If no — if the whole task is "read these inputs, produce this output" — it is one-shot generation. Do not build an agent. You will spend 4x the effort for 20% more capability.

If yes — the task requires fetching data, calling APIs, writing to systems — continue.

3. Is the value of automating the sequence greater than the cost of reviewing it?

This is the most important question.

If a human would do the task in 90 seconds and the agentic version is right 85% of the time, the net savings are negative — reviewing a wrong agent is more expensive than doing it yourself.

If the task takes 45 minutes and the agent is right 85% of the time with fast review, the math works — the 15% error rate is cheap to catch and the 85% saved is real.

Most workflows don't pass this test. People assume they do.

4. Does failure mode cost matter more than success rate?

This is the one that kills a lot of agentic enthusiasm. If the cost of a wrong action is catastrophic — an incorrect trade, a wrongful denial, a missed sanctions hit, a deleted client file — the agent doesn't belong in the loop without a human approval gate at the consequential step, regardless of success rate.

If you put the approval gate in, you have an agent-assisted workflow, not a fully agentic one. That's fine. It's usually what you want.

Applying the tree to real workflows

A few worked examples from the SZL portfolio.

Port-call extraction (Vessels):

- Fixed structured output. One-shot, not agentic.
- Decision: one-shot generation with eval-gated prompts and strict validation.

Pipeline screening against investment thesis (Terra):

- Fixed structured output (score + reasoning).
- Decision: one-shot generation. Operator reviews the score.

Client intake (Prism Counsel):

- Multi-step: classify, route, assign, notify.
- Sequence automation worth it; each step low-consequence individually.
- Decision: agentic loop with no per-step approval.

Contract redlining (Prism Counsel):

- Multi-step: parse, compare, suggest, apply.
- High-consequence (legal effect).
- Decision: agent-assisted, not agentic — lawyer approves every redline.

Investor update draft (internal):

- Open-ended output.
- Multi-source synthesis.
- High-consequence (relationships).
- Decision: one-shot draft + human heavy edit. Not agentic.

Daily cross-domain brief (Command Portal):

- Fixed structure (top signals, top decisions).

- Multi-source synthesis.
- Low-consequence (informational).
- Decision: one-shot generation, refreshed hourly. No agent needed.

The meta-lesson

"Should this be an agent?" is almost always the wrong first question. The right first question is: "what's the smallest AI shape that makes this workflow 3x better?" One-shot generation is usually the answer. Agents are the last resort, not the first swing.

When you do build an agent, scaffold it as ruthlessly as possible: tool allowlists, budget caps, step limits, human approval gates at consequential actions, full trace logging. The reason agents fail in production isn't that models aren't capable enough. It's that teams skip the scaffolding.

Next post: the state of the maritime commercial market, from inside Vessels.

— S

The five questions, in order

Before any workflow becomes an agent, it answers all five.

One: is the workflow well-defined? If a senior operator can write the SOP in under a page, the workflow is well-defined. If it takes ten pages with branching exceptions, the workflow is implicit, and turning it into an agent encodes the wrong process. Fix the SOP first.

Two: is the cost of being wrong bounded? Agents make mistakes. The question is how expensive each mistake is. A misclassified support ticket is cheap. A misrouted weapons-release recommendation is not. Cost-bounded workflows are agent candidates. Cost-unbounded workflows stay assistive.

Three: can the work be evaluated? If you cannot write the eval — the test that says "this output is correct, this output is not" — you cannot operate an agent in production. The eval is the contract. No eval, no agent.

Four: is there enough volume to justify the engineering? Agents take 4–8 weeks of engineering and another 4–8 weeks of evals and guardrail work to harden. If the workflow runs ten times a year, the math doesn't work. Agents need volume.

Five: is there a human stop in the chain? Even the best agents need a human approver on irreversible actions. If the workflow can't accommodate a human-in-the-loop checkpoint without becoming useless, the workflow is too autonomous to ship safely today.

What we ship as agents

Inside SZL, four workflows pass the five tests right now.

Inbox triage — sorting customer email into product-, advisory-, or press-related queues with a two-line summary attached. Eval is straightforward: the recipient either keeps the routing or corrects it. Volume is high enough.

Contract first-pass redlines — Prism Counsel produces them and a lawyer reviews. Bounded cost, clear eval, human-in-the-loop on send.

Port-call anomaly detection in Vessels — the agent flags vessels whose AIS pattern deviates from expected behavior, and an analyst confirms. Cost of false positive is bounded; cost of missing a real event is what the analyst is paid to catch.

Pull-request first-pass review for our internal codebase — the agent reviews diffs against our style and security guidelines and posts comments. The human reviewer still owns the merge decision.

Everything else — and there are at least twenty other workflows we've considered — is assistive AI inside the operator surface, not an autonomous agent. The bar for "this runs without a human in the loop" is high, and that's the right place for the bar to be.

SUBSTACK TAGS: AI AGENTS PRODUCT

MEDIUM TAGS (PICK 5): ARTIFICIAL INTELLIGENCE AGENTS PRODUCT DESIGN
SOFTWARE ENGINEERING AUTOMATION

State of the Commercial Maritime Market, Q1

Freight, bunkers, compliance pressure, and what operators are actually doing about it.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

From inside Vessels I get a useful view into what's actually happening in commercial maritime, across tanker, dry bulk, and container segments. Here's the Q1 picture.

Freight rates: bifurcation, not a cycle

The headline across the press is "rates are mixed." The truer statement is: the market is bifurcating. Vessels serving constrained routes — the ones where political risk, port congestion, and environmental regulation stack up — are earning a premium that didn't exist two years ago. Vessels on commoditized routes are earning what they earned in the last down-cycle.

The practical implication for operators: route choice is the largest single variable in this year's P&L, by a wider margin than at any point in the last decade. Vessels' voyage P&L module and rate benchmarking surface exist because this variable has gotten too large for spreadsheet analysis.

Bunker economics

Three things happening at once:

- **Compliance fuel mandates** are driving a real premium on low-sulfur and emerging alternative fuels. The gap between cheapest-compliant and cheapest-available is the largest it has been.
- **Bunker supplier reliability** is more variable than headline price suggests. Delivery delays and quality disputes are up.
- **Emissions accounting** is now a cost line, not a future risk. EU ETS is real money. FuelEU Maritime is about to be.

Operators who still source bunkers on phone relationships and spreadsheets are, on average, paying 3–7% more than operators using systematic optimization. That's a margin problem on thin-margin voyages.

Compliance pressure

Maritime compliance has moved decisively into the board conversation. The specific regimes mattering right now:

- **EU ETS** — phased-in, real costs, heavy reporting.
- **FuelEU Maritime** — phased-in starting this year, complex accounting.
- **CII** — rating-based, reputational impact growing.
- **Sanctions regimes** — expanding scope, enforcement attention up materially.

- **MRV / IMO DCS** — data quality now audited, not assumed.

Operators who haven't invested in a compliance command surface are doing this work in spreadsheets, with audit trails that will not survive a serious review. The buyers who were "not yet ready" for compliance tooling 18 months ago are ready now.

Counterparty quality

A quieter story: counterparty quality is more variable than a few years ago. Port agents, bunker suppliers, and small-fleet owners are under margin pressure. Dispute frequency on port calls is up. Payment delays on chartering are up. Insolvency risk in specific sub-segments is up.

The commercial decision this forces: operators are consolidating counterparty relationships and demanding better transparency. Vessels' counterparty-risk surface — which lets operators see the full history of performance, disputes, and financial signals on a given counterparty — is a surface that did not exist in a usable form in the market 24 months ago. That's a wide-open opportunity.

What operators are actually doing

Across the three segments Vessels serves, a few consistent moves:

- **Investing in structured data capture** of port calls and voyages. The operators who lagged here are now playing catch-up urgently.
- **Consolidating vendor stacks.** The appetite for seven separate systems held together by Outlook is gone. RFPs increasingly ask for unified surfaces.
- **Hiring differently.** Ex-tech talent moving into operations roles, bringing product discipline. Fleet operations centers that looked like trading desks 18 months ago look more like SRE rooms now.
- **Rebuilding the chartering function** around better data. The best commercial teams now run on rate benchmarking and counterparty risk as inputs — not just instinct.

What I think happens next

Three predictions for the remainder of the year, confidence levels attached.

1. **Rate bifurcation widens** (high confidence). Premium routes earn more; commoditized routes compress further. Route selection becomes the dominant P&L variable.
2. **Compliance command surfaces become non-optional** (high confidence). By end of year, every serious operator will have a procurement process for compliance tooling. The market is wide open.
3. **First large-scale maritime software rollup** (medium confidence). Vendor consolidation in maritime software is due. The incumbents are fragmented and slow; the new entrants — including Vessels — are credible. Expect M&A activity.

If you're an operator and this reads right or wrong, reply. These posts compound when people push back. Vessels is built on conversations with operators, not product strategy sessions.

Next post: what I learned from the one pilot we lost in Q1.

— S

Five forces shaping the quarter

One: Red Sea reroutes are now structural. Container traffic around the Cape of Good Hope is no longer a tactical detour — major liners have rebuilt their schedules around the longer route. That adds 10–14 days per Asia–Europe rotation, which has tightened effective vessel supply by roughly 5–8% on those trades. Charter rates on relevant tonnage are up accordingly.

Two: bunker prices are diverging by region. Singapore VLSFO is trading at a meaningful discount to Rotterdam VLSFO again, and the differential is wide enough to influence routing decisions for tankers that have flexibility. Operators with bunker-optimization tooling are capturing the spread; operators without it are leaving it on the table.

Three: EU-ETS bills are landing. The first invoices for 2024 emissions are arriving, and the bills are larger than several charterers had modeled. Disputes between charterers and owners over who pays — and on what basis — are the dominant subject of fixture-note negotiations this quarter.

Four: sanctions enforcement is tightening on shadow-fleet operators. Dark-fleet tonnage moving Russian crude has had a soft regulatory environment for two years. That's changing — flag-state pressure, P&I withdrawals, and port-state enforcement are all up. The shadow fleet is not going away, but the operating cost is rising.

Five: dry-bulk is quietly recovering. After eighteen months of soft conditions, Capesize earnings have moved from break-even to comfortably profitable on the back of stronger Brazilian iron-ore exports and stable Chinese demand. The recovery is not yet pricing into newbuilding orders, which is unusual.

What this means for operators

Three operational implications.

Tighten your laytime and demurrage tracking — with ETA volatility this high, the difference between disciplined and casual ops on demurrage is six figures per voyage. Vessels surface this for our customers, but the discipline is what produces the recovery, not the tool.

Re-paper your charter parties for EU-ETS allocation. The standard BIMCO clause is probably already in your fixtures, but the practical allocation between owner and charterer needs to be unambiguous. Disputes here will dominate Q2.

Stress-test your counterparty exposure. With shadow-fleet enforcement tightening, secondary sanctions risk is up. If your chain includes a flag, an insurer, or a charterer that has any exposure, find out now, not when the office of foreign assets control finds out for you.

SUBSTACK TAGS: MARITIME MARKET COMMERCIAL

MEDIUM TAGS (PICK 5): MARITIME LOGISTICS SHIPPING MARKET ANALYSIS

SUPPLY CHAIN

What I Learned From the One Pilot We Lost

A structured post-mortem on the engagement that didn't make it to production, and what I'd do differently.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

In my Q1 investor update I mentioned one pilot that churned in month two of a three-month engagement. I promised a full post-mortem. Here it is.

The facts

- Customer: a mid-market operator in one of SZL's core verticals. I'm going to stay vague on segment to protect them.
- Product: one of the SZL domain platforms.
- Engagement: three-month paid pilot with defined success metrics and a production-rollout path.
- Outcome: customer exited in month two. Full refund offered; they accepted a partial credit toward a future engagement.
- Reason cited: internal reorganization, champion changed roles, new champion did not carry the relationship forward.

The honest diagnosis

The reason they cited was real. It was also not the whole story.

Here's the fuller diagnosis, in the order I learned it:

- 1. The champion was singular.** One person inside the customer had the vision, the credibility, and the will to bring the pilot in. When they moved, the engagement had no second anchor. That's a pattern. I've started flagging "single-champion pilots" as a risk tier from day one.
- 2. The executive sponsor was nominal.** On paper we had a sponsor two levels above the champion. In practice they had signed because the champion asked them to, not because they had their own belief in the project. When the champion left, the sponsor defaulted to the new champion's priorities, which were different.
- **3. Success metrics were measured but not *celebrated*.**** We hit the month-one milestones. The champion was pleased. The sponsor was not in the celebration loop. A one-paragraph email to the sponsor at each milestone would have cost nothing and might have changed the outcome. I skipped it. My fault.
- 4. The contract did not include a champion-succession clause.** In enterprise paper this is increasingly standard: if the main contact changes roles, the supplier gets a meeting with the new contact before any renewal or cancellation decision. Would I have saved the engagement with that clause? Maybe not. Would I have had a better shot? Yes.

5. I over-invested in product fit, under-invested in political fit. The product was genuinely good for this customer. I could see the ROI. I convinced myself the product's quality would carry political risk. It won't, ever. Political fit is a separate problem and has to be solved separately.

What I'm changing

Four structural changes, starting with the next pilot we sign.

- 1. Two named champions, not one.** Every pilot requires identifying a second internal champion in the first 30 days. If we can't, we flag it as a risk tier and manage accordingly.
- 2. Executive sponsor has their own success metric.** Not just the champion's metric. Something the sponsor personally cares about — a business outcome that would make them look good in their next quarter review. They own it. We report to them on it separately.
- 3. Milestone celebration is a structured artifact.** Every milestone now ships with a one-page ROI memo written for the sponsor's audience, not the champion's. Email, not meeting. Cost: 20 minutes. Payoff: large.
- 4. Champion-succession clause in the paper.** Standard from now on. If the key contact changes, we get a 30-day window and a meeting.

What I'm not changing

I am not changing the pilot pricing. The pilots are priced to qualify commitment, not to hedge churn risk. If a customer isn't willing to put meaningful money on the table for a three-month engagement, we shouldn't be doing the pilot regardless of political dynamics.

I am not changing the success-metric discipline. The metrics are the right metrics. We hit them. The loss was not a metrics problem.

I am not shortening the pilot. Three months is the right length for this class of product. Shorter pilots are demos with invoices attached.

The meta-lesson

Pilots die of political problems more often than product problems. Founders — especially technical founders — systematically underweight this. The five structural changes above are cheaper than losing the next pilot for the same reason.

If you run pilots in your business, stress-test every one of them against a single-champion-leaves scenario. If the answer is "the engagement dies," you have more work to do on political fit. That work is boring and it compounds.

Next post: the CORTEX mobile story.

— S

What actually happened

Three things, none of them surprising in retrospect.

The buying committee changed mid-pilot. Our champion was a director of operations. Six weeks into the pilot, that director took a job at a competitor, and the new director arrived with her own preferred vendor relationship from a previous role. We had not built relationships with the rest of the committee. When the champion left, our coverage was zero.

We over-indexed on technical proof and under-indexed on procurement readiness. Our pilot data was excellent. The customer's procurement team had questions about insurance coverage, security certifications, and data-residency that we had not pre-answered, and answering them under deadline pressure looked sloppy. Procurement teams say no when they're surprised; we surprised them.

We failed to make the pilot success criteria binding. The kickoff defined success as "a 30% reduction in time-to-classify across the queue." We hit 41%. The customer's view at the end was that the reduction was real but "not transformative enough to justify the implementation cost." There was no contractual or even verbal definition of what would have been transformative enough. We had built a great answer to an unwritten question.

The three changes we made

Two-named-champion rule. Every pilot now requires a named primary champion and a named secondary champion at signature. Both must be in the kickoff. If either departs the organization, we have an automatic check-in with the other and the pilot owner.

Pre-flight procurement memo. Every pilot kickoff now includes a one-page document that lists every standard procurement question we know procurement teams ask — insurance, certifications, DPA terms, data-residency, SLA, exit clause — and our pre-written answer to each. Procurement teams open the memo, find their question already answered, and move on. The pilot stops getting blocked at week ten by a question that should have been answered in week one.

Quantified success criteria with conversion-trigger language. "If the pilot achieves X, the customer agrees to convert to a paid annual contract on the terms in Annex A. If the pilot does not achieve X, the customer agrees to provide a written description of why and to consider a follow-on engagement." Both sides know what they're agreeing to. Pilots stop being open-ended evaluations and become structured commercial trials.

The pilot we lost was a million-dollar lesson. Cheaper than learning it on the fifth pilot, which is when the same failure mode would have cost us materially more reputation.

SUBSTACK TAGS: [FOUNDER](#) [LESSONS](#) [POST-MORTEM](#)

MEDIUM TAGS (PICK 5): [STARTUP](#) [LESSONS LEARNED](#) [ENTREPRENEURSHIP](#)

[BUSINESS STRATEGY](#)

[PRODUCT MANAGEMENT](#)

CORTEX: Why Command Platforms Need a Mobile Surface

Operators don't stay at a desk. The mobile command app is where the next generation of trust lives.

SZL COMMAND • SZLCOMMAND.SUBSTACK.COM • COPY-PASTE READY

Most enterprise software still assumes the operator is at a desk. Command platforms especially. That assumption is wrong for the users who matter most.

CORTEX is SZL's mobile command surface. This post is why it exists and what it does.

Who actually uses command platforms

Spend time with real operators — a port captain, an intel analyst, a real-estate asset manager, a GC — and a pattern shows up. These people are not at their desks most of the day. They're on a vessel, on a site, in a meeting, in a car, in transit.

The enterprise software they use at their desk is locked to the desk. Their phone has email and a messenger. The signal that matters flows through the messenger because the software can't meet them on the phone.

This is a product opportunity, not a product accessory. Put the command platform on the phone *correctly* and you become the messenger. Put it there badly and you're just another push notification competing for attention.

What CORTEX is

CORTEX is the mobile surface for every SZL command platform. Four design commitments:

- 1. It's not a miniaturized dashboard.** Shrinking the web app onto a phone is the wrong answer. CORTEX is its own surface with its own information design, optimized for phone-sized decisions.
- 2. It's the alert layer.** Cross-domain alerts from Command Portal route to CORTEX. Push notifications are structured, prioritized, and deep-link to the exact workspace the operator needs.
- 3. It's the approval layer.** Every agentic workflow in the portfolio that needs a human approval gate routes that approval to CORTEX. Sign-offs happen in seconds on the phone, not hours at a desk.
- 4. It's the daily brief surface.** The cross-domain morning brief — what happened across every product overnight — is designed for CORTEX first and the web second.

The hard parts

Three I've hit.

Identity across domains. The same authentication model that makes Command Portal coherent has to carry to mobile, with biometric fallback, device trust, and compartment-level access for sensitive domains. This is harder than web auth by a margin that I underestimated by a factor of three.

Notification discipline. Push notifications are a trust budget. Spend it wrong and the operator mutes the app within a week. Every CORTEX notification is classified — urgent, important, informational — and the defaults are conservative. Operators can tune thresholds per product.

Offline-capable design. Operators lose connectivity. CORTEX has to handle graceful degradation — show the last-known state, queue the action, sync when back online. This is a whole class of engineering problem that web-first products don't have to solve.

The bet

The bet is simple: operators who run their work on a phone will stay with the product that serves them on the phone well. Every SZL command platform has a CORTEX surface. No operator has to ask "does this work on my phone?" The answer is always yes, and the mobile surface is not an afterthought — it's the first-class surface for the decisions operators make while mobile.

CORTEX is live, in internal use across the portfolio, in external beta with two design partners in maritime and real estate. I'll write a dedicated post on the mobile-first UI patterns we landed on after this — the design side of CORTEX deserves its own essay.

Next post: the 90-day review — what I got right, what I got wrong, what's next.

— S

What CORTEX is, in one sentence

CORTEX is the read-and-act mobile surface for the SZL portfolio — the screen an operator opens at 6am or 11pm to see what changed and to take a small number of high-trust actions.

What it is not

CORTEX is not a thinned-down version of every product on a small screen. That pattern fails. The desktop product has hundreds of screens; the mobile product needs maybe twenty. The discipline is not "what can we fit on mobile" — it's "what does the operator actually need to do on mobile."

For each product, the answer is small.

Aegis on mobile — see the active alerts queue, acknowledge an alert, request analyst follow-up, view the COP at low fidelity, push an urgent message to the team. Five actions.

Vessels on mobile — see the fleet status, drill into a single voyage, approve or reject a fixture-note, contact the broker, log a port-call exception. Five actions.

Terra on mobile — pull a quick comp set on a property, view the underwriting summary, approve a routine LOI, share a market brief with a partner. Four actions.

Carlota Jo on mobile — view the active engagement list, approve a deliverable, message a client, log billable time. Four actions.

That's eighteen actions, total, across the four primary surfaces. Everything else stays on desktop, where the screen is bigger and the input is a keyboard.

Why mobile matters now, not later

Two reasons.

First, the operators we serve are not at desks all day. A program-office officer in Aegis is in briefings. A charterer in Vessels is on the trading floor or on a call with a broker. A real-estate principal in Terra is touring properties. The 80/20 of high-leverage moments — approve, reject, acknowledge, escalate — happens on a phone. If the product can't capture those moments, the desktop product becomes a system of record after the fact, not a system of action.

Second, mobile is where trust gets built or lost in seconds. If the operator opens CORTEX at 6am and the data is twelve hours stale, or the latency is bad, or the layout is confusing, they stop opening it. And if they stop opening it, they stop trusting the desktop product too — because they assume the desktop is the same surface in a bigger window. CORTEX has to be best-in-class mobile software, not "the mobile app."

We're shipping CORTEX in stages — Aegis read-only first, then Aegis read-and-act, then Vessels, Terra, and Carlota Jo each rolled in over the following two quarters. By end of year, every operator role inside the SZL portfolio has a tuned mobile surface. That's the bet.

SUBSTACK TAGS:

CORTEX

MOBILE

PRODUCT

MEDIUM TAGS (PICK 5):

MOBILE

PRODUCT DESIGN

ENTERPRISE SOFTWARE

UX

TECHNOLOGY

90 Days of Building in Public

The review. What I got right, what I got wrong, and what's next.

SZL COMMAND · SZLCOMMAND.SUBSTACK.COM · COPY-PASTE READY

Ninety days ago I started writing this newsletter in public, with a thesis, a calendar, and a set of promises. This is the review.

I promised to publish twice a week. I did, with one miss. I promised the investor update would be public. It was. I promised I'd name the things I got wrong as openly as the things that worked. I did, including a full post-mortem on the pilot we lost.

Here's what changed.

What the newsletter changed

Inbound doubled. Partnership conversations, pilot conversations, investor conversations, hiring conversations. Not all qualified — much more than before. The writing pre-qualifies. People who read four posts and then write in already understand the thesis.

Relationships got deeper, faster. The calls I'm on now start at a different place. Instead of explaining what SZL is, I can spend the hour on the specific thing that matters to the counterparty. The newsletter does the explaining.

Roadmap got sharper. The act of writing about a product every few weeks forced me to see the product from the outside. Three roadmap decisions changed materially because I could not defend the previous version in a public essay.

The team writes better. This one surprised me. Seeing the founder write in public gave the rest of the team permission to write — internal memos, client-facing docs, roadmap explanations. Writing quality across the org went up.

What I got right

Two posts a week was the correct cadence. Enough to compound, not so much that the writing degrades. Every time I've been tempted to go to three, I've resisted.

Pillar structure held. Six pillars sounded like a lot at the start. In practice it's the right number — enough variety that the newsletter doesn't get monotonous, tight enough that the brand stays coherent.

Cross-posting to Medium paid off. Medium's long-tail distribution delivered roughly 25% of new subscriber growth after month two. Free distribution; the cost is 15 minutes to adapt each post.

Publishing the investor update publicly was the right call. The piece that most people told me was risky did more for fundraising conversations than the most polished deck I've made. Candor compounds.

What I got wrong

I underestimated paid conversion. I assumed a 3% paid conversion rate; actual was 7% inside the first 60 days, which is a different business. I should have launched the paid tier in week two, not week eight. Left meaningful money on the table.

I under-invested in visual direction. The newsletter has lived mostly on typography, which works but undershoots. The 90-day mark is the right moment to bring more design discipline — custom hero illustrations per post, a tighter look on the cover image.

I over-rotated on founder-journey posts. Pillar six was meant to be a rotation, not the backbone. A few weeks it became the default, and the engagement data showed readers wanted more product and market pieces. Re-balancing.

I was slow to interview. Guest voices — operators, writers, investors — would have deepened the newsletter faster. One interview a month going forward.

What's next

The next 90 days:

- **Launch the paid tier properly.** Founding-member pricing open for a capped window. Quarterly investor memo for paid members. Monthly office-hours call.
- **Re-balance pillars.** Founder-journey drops to 25% of posts, product/market takes the rest.
- **Start interviews.** One per month, long-form.
- **Ship a custom design system for the newsletter.** Hero illustrations, better post layout, on-brand downloads.
- **Open a private forum for paid subscribers.** Small, curated, with office hours and a directory. This is where a lot of the relationship value compounds.

The thank-you

If you've read along — all of you, in every pillar and every product arc — thank you. The newsletter would not work without the replies. The replies turned into roadmap decisions, hires, partnerships, and in one case an entire product direction change.

The work is early. The thesis is holding. The next 90 days matter more than the last.

Keep reading. Keep replying. Forward this to one person who should be in the room.

— S

The numbers, exactly

Subscribers. 8,420 free, 312 paid, 47 founding members. Above the 90-day plan on free; below on paid; ahead on founding members.

Open rate. 47% on essay sends, 58% on the monthly roundup. Above target on both.

Paid conversion. 3.7% within 60 days of subscribing. Below the 5–8% target. Diagnosis underway.

Inbound qualified conversations. 67. Above the 40 target. Of these: 14 product pilots in active discussion, 9 advisory engagements scoped, 6 investor conversations. Six closed (three pilots, two advisory, one strategic intro).

Press coverage. Three industry publications picked up essays for syndication or quoted them. Two podcast appearances scheduled for next quarter on the strength of the newsletter.

Sales-cycle compression. From 84 days to 61 days. Below the 30-day target but trending the right direction. Most of the compression came from pre-publishing legal artifacts via Prism, not from being more aggressive in conversations.

What worked

The cadence held. 24 essays published, zero missed Tuesdays or Fridays. The discipline of writing two essays a week is the single highest-leverage thing I do, and it isn't close.

The pillar mix calibrated quickly. Defense and maritime essays drove the most product inbound. AI engineering essays drove the most subscriber growth. Founder-journey essays drove the most replies. Knowing this changes the calendar for Q2.

The founding-member offer over-performed. We capped at 100 and hit 47 in 90 days without pushing. Closing the cap to 75 in Q2 to preserve the scarcity that made the offer attractive in the first place.

What didn't

The newsletter-to-paid conversion is too low. The diagnosis: I waited too long to ask for the upgrade and I priced the founding-member tier under what the audience would have paid. Both fixable. Q2 will reset the conversion approach with a clearer ask in essays 25 and 30, and a price increase on the standard paid tier from \$15/mo to \$20/mo (founding-member price holds for the existing 47 indefinitely).

The cross-posting ROI on Medium is below model. Medium views are healthy but conversion to Substack subscribers is a fraction of what LinkedIn produces. We will keep cross-posting because the SEO value is real, but I'm reweighting the time investment toward LinkedIn and X.

What's next

90 days of the same cadence, with three changes. A clearer monetization ask in two essays per quarter. A price reset on the standard paid tier. And one new pillar essay format — quarterly state-of-market pieces, syndicated to industry publications, designed to make SZL Command the citable source on each of our verticals.

The bet was that publishing in public would compound trust, distribution, and pipeline faster than not publishing. 90 days in, the bet is paying. The next 90 days are the second half of the test.

SUBSTACK TAGS:

FOUNDER

REVIEW

REFLECTION

MEDIUM TAGS (PICK 5):

BUILDING IN PUBLIC

STARTUP

ENTREPRENEURSHIP

REFLECTION

LEADERSHIP

Growth plan.

Cross-posting strategy

- Primary publication on Substack with email-owned list and paid tier.
- 24-hour-delayed cross-post to Medium with canonical URL back to Substack to protect SEO and email metrics.
- LinkedIn short-form (150–250 words) same-day with a single branded image and a CTA back to Substack.
- X thread (6–10 posts) same-day with the sharpest claims from the essay and a link in the last post only.
- Quarterly deep-cut posts go to the SZL Holdings site blog as pillar content for SEO.

Amplification rhythm

- Founder X/LinkedIn accounts post every essay same day within an hour of Substack publication.
- Team members (when they join) get a curated share template — no spammy comment swaps.
- Design partners and customers get a heads-up 24 hours before any essay that names them.
- Each month, one essay is picked as a 'flagship' and pitched to a Medium publication (The Startup, Towards AI, domain pubs).
- Every essay ends with one concrete ask — reply, intro, subscribe — never three.
- Monthly roundup email to subscribers only: the month's essays, the most replied-to, and upcoming topics.

GitHub / portfolio wiring

- Every GitHub repo README links to the relevant SZL Command essay for context.
- Cross-link essays that reference open-source work from inside the essay.
- A dedicated 'Reading' section on the SZL Holdings org README points to the newsletter.

Partner cross-promos

- Adjacent-vertical newsletters (maritime, defense, real-estate, AI infra) — three curated cross-promos per quarter, no trade-for-trade spam.
- Guest essays: one per month from month three, targeting complementary audiences.
- Podcast appearances: one per month from month two, prioritizing investor and operator audiences over pure tech shows.
- Conference and industry-event mentions: speaking slots routed back to the newsletter as the archive.

Paid tier pricing

Free: All essays, monthly roundup.

Paid: \$15 / month or \$150 / year (standard). Includes quarterly investor-style memo, monthly office-hours call, private forum and directory.

Founding member: \$500 / year (capped at 100 seats in year one). Everything above plus a 1:1 call with the founder per year and early previews of portfolio product roadmaps.

Launch offer: Founding-member price locked for three years if signed up in the first 90 days.

LinkedIn & X templates.

LinkedIn short-form template

[Hook – the sharpest claim from the essay, 1 sentence]

[2-3 short paragraphs, one idea each, distilling the argument. Use single-line paragraphs for whitespace.]

[Optional: one concrete example or numeric anchor.]

Full essay on SZL Command (free to subscribe) → [link]

#[primary tag] #[secondary tag] #[narrow tag]

X thread template

Post 1 (hook): [1-sentence sharpest claim]

Post 2: [the problem, 1 sentence]

Post 3: [the insight, 1 sentence]

Post 4: [first supporting point]

Post 5: [second supporting point]

Post 6: [third supporting point / specific example]

Post 7: [meta-lesson / "why this matters"]

Post 8 (last, contains link): [recap + link to SZL Command essay]

Rules:

- One idea per post.
- No paragraph breaks inside a post.
- Link only in the last post.
- Images on post 1 only.

Monetization plan.

Free vs paid

Free tier: All essays, the welcome email, the monthly roundup, and the public Q-update.

Paid tier includes:

- Quarterly investor-style deep-dive memo (paid-only).
- Monthly small-group office-hours call (45 min, capped at 20 attendees, paid-only).
- Private forum with the paid subscriber directory.
- Quarterly 'ask-me-anything' thread in the forum.
- Early access to paid-tier-only product previews across SZL Holdings.

Founding-member offer

Price: \$500 / year

Cap: First 100 seats, then closed until year two.

- All paid-tier benefits.
- One 1:1 call with the founder per year (30 min).
- Early previews of portfolio product roadmaps.
- Founding-member badge on the forum.
- Founding-member pricing locked for three years.

Consulting funnel tie-in

The newsletter is the top of funnel for Carlota Jo's advisory practice. Any reader who identifies as an operator, GP, program office, or executive can reply to any essay and ask about an advisory engagement. Three qualification filters run on every inbound before a call is scheduled:

1. **Scope fit** — does the engagement need match Carlota Jo's practice areas (AI platform strategy, vertical command-surface design, regulated-industry governance)?
2. **Budget fit** — is the client at a stage where advisory spend is appropriate (\$100K+ engagements)?
3. **Timing fit** — can we deliver quality in the client's timeline without compromising the SZL product roadmap?

If all three pass, the next step is a 30-minute scoping call. Proposals ship within five business days.

Revenue mix target

STREAM	YEAR 1	YEAR 3
Newsletter paid	5%	3%
Product pilots & ARR	60%	80%
Carlota Jo advisory	30%	15%
Other	5%	2%

KPIs & tracking sheet spec.

Subscriber targets

MILESTONE	FREE	PAID
Month 1	1,500 free / 40 paid	
Month 3	5,000 free / 250 paid	
Month 6	12,000 free / 700 paid	
Month 12	30,000 free / 2,000 paid	

Engagement targets

- Open rate:** 45% on main essays, 55% on monthly roundup. Minimum: 35% — below this, diagnose list hygiene and subject lines.
- Clickthrough:** 8–12% clickthrough to linked essays or archive.
- Free→Paid conversion:** 5–8% free-to-paid within 60 days of subscribing.
- Paid churn:** <2% monthly on paid.

Inbound pipeline targets

- Month 1:** 10 qualified conversations (pilots, advisory, investors).
- Month 3:** 40 qualified conversations.
- Month 12:** 200 qualified conversations; at least 10% converted to paid engagements or investments.

Tracking sheet columns (paste into Google Sheets)

```
post_id publish_date    pillar  title    substack_opens  substack_open_rate
substack_clicks substack_new_subs      medium_views_7d medium_views_30d
medium_reads    medium_fans    medium_read_ratio    linkedin_impressions
linkedin_clicks linkedin_comments    x_impressions    x_clicks    x_replies
paid_conversions    inbound_replies qualified_conversations
```

Cadence: Updated weekly, every Monday morning, from the four platform analytics dashboards. Quarterly roll-up feeds the public investor update.

Substack setup checklist.

Step-by-step launch checklist. Follow in order.

STEP 01 • Reserve the publication handle

Create the Substack publication at szlcommand.substack.com (or the chosen handle). Set publication name to 'SZL Command'.

STEP 02 • Upload the brand assets

Logo (gold S-mark on obsidian), favicon, and cover image. Use [mockups/substack-home.png](#) from the screenshots zip as the visual target — the gold brand mark in the top-left of that mockup is the SZL Command logo. Crop the 32×32 mark for the favicon and the hero gradient for the publication cover.

STEP 03 • Set the tagline and bio

Tagline: "Command platforms, written in public.". Bio: use the short bio from the positioning section. Use the Inter/Space Grotesk pairing for any custom embedded designs.

STEP 04 • Write the About page

Paste the About-page copy from the positioning section verbatim. Add a headshot or a branded monogram.

STEP 05 • Configure the welcome email

Use the welcome email copy from this document. Subject, preview text, body all provided.

STEP 06 • Configure paid tier

Standard: \$15/mo, \$150/yr. Founding: \$500/yr capped at 100 seats. Add founding-member benefits copy from the monetization section.

STEP 07 • Set up custom domain (optional but recommended)

Point szlcommand.szlholdings.com (or similar) to Substack via CNAME. Do this before launch to avoid URL fragmentation later.

STEP 08 • Publish the first three posts as a staggered launch

Post 1 (Why I'm Building SZL Holdings in Public) at launch. Post 2 (The Case for Vertical Command Platforms) 3 days later. Post 3 (Inside Aegis) 4 days after that. Front-load the launch week with the strongest posts to seed subscriptions.

STEP 09 • Send launch announcement to personal network

Personal email to 50–100 people (investors, ex-colleagues, friends of the firm) with a one-paragraph note and the first-post link. Do not blast — personalize.

STEP 10 • Amplify on LinkedIn and X

Use the LinkedIn short-form and X thread templates from this kit for each of the first three posts.

STEP 11 • Submit to Substack recommendations

Reach out to 5–10 aligned Substack publications (Stratechery-adjacent, vertical AI, founder-voice) and request mutual recommendations. Be specific about why the audience fit works.

STEP 12 • Cadence lock

Lock Tuesday 09:00 ET and Friday 09:00 ET publishing times in the calendar for the next 90 days.

STEP 13 • Analytics setup

Copy the tracking sheet spec into a Google Sheet (or Notion database). Pull the first week's metrics manually to validate before automating.

STEP 14 • Paid tier public launch

Week 2 or 3 post-launch, not before. A 'what's in the paid tier' post on the free list, with the founding-member offer and cap clearly communicated.

STEP 15 • 90-day review

At day 90, publish the '90 Days of Building in Public' post (included in this kit). Use the KPI tracker to back the numbers.